

Article

Semantic-Driven Context Pruning for Arabic RAG: Quality-Preserving Compression Under Token Inflation

Ali Abdul Razzaq Tarish¹ and Noor Abdul Khaleq Zghair^{1,*}¹ College of Computer Engineering, University of Technology - Iraq, Baghdad, Iraq

* Correspondence: Noor.A.Zghair@uotechnology.edu.iq

Abstract

Retrieval-Augmented Generation (RAG) for Modern Standard Arabic (MSA) faces a memory bottleneck: Arabic's morphology inflates token counts relative to English, enlarging the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must hold per request. We introduce a Lightweight Semantic Pruning Middleware (LSPM) that scores retrieved passages at the sentence level with a cross-encoder and retains only the most query-relevant subset. Two tokenizers confirm Arabic requires 1.72–1.79× as many tokens as English for equivalent content (30 pairs, $p < 0.0001$). On 140 Arabic reading-comprehension questions in a controlled six-document candidate pool (one guaranteed gold passage plus five distractors; Section 3.4), LSPM significantly outperforms naive sentence-count-matched truncation on token-F1 at the most aggressive ratio tested (F1 0.652 vs. 0.537, Holm-adjusted $p = 0.0067$) and remains equivalent to the unpruned answer; the advantage replicates under a token-budget-matched rerun, against LLMingua-2, and on an independent fresh-sample replication. A repeated GPU benchmark scoped to generation over an already-pruned context and excluding the pruning step's own ~506 ms cross-encoder scoring cost (quantified separately in Section 3.1) finds no measurable throughput penalty at low KV-cache occupancy; a higher-pressure ablation reaches 99.2% occupancy and confirms the dynamic controller adapts, though not that adapting beats a matched fixed ratio, so the memory-efficiency claim remains analytical. A true end-to-end benchmark that instead scores and prunes context inside the live request path finds a substantial, concurrency-dependent throughput penalty for LSPM, and a matched controller-versus-fixed-ratio comparison under memory pressure is a mixed result with no answer-quality data collected. We release the implementation, evaluation code, and datasets described in this paper.

Keywords: retrieval-augmented generation; Arabic natural language processing; prompt compression; vLLM; large language models

Academic Editor: Firstname
Lastname

Received: date
Revised: date
Accepted: date
Published: date

Copyright: © 2026 by the authors.
Submitted for possible open access
publication under the terms and
conditions of the [Creative](#)

1. Introduction

Large Language Models (LLMs), built on the Transformer architecture [1], have become the default reasoning engine behind Retrieval-Augmented Generation (RAG) systems that ground generation in externally retrieved evidence rather than relying solely on parametric memory [2]. The scaling of Transformer-based decoders to tens of billions of parameters, demonstrated at scale by GPT-3's few-shot capabilities [3], is precisely what makes every additional context token expensive: larger models carry larger per-token Key-Value state, so the tokenization inefficiencies this paper measures for Arabic compound directly with model scale. Every retrieved token that enters an LLM's context window occupies a slot in the Key-Value (KV) cache that memory-efficient serving engines such as vLLM must hold in GPU memory for the duration of the request [4]. vLLM's PagedAttention algorithm manages this pressure by paging the KV cache into fixed-size blocks [4], and related techniques such as attention-sink streaming [5] and heavy-hitter eviction [6] further manage cache consequences after tokens have already entered it. None of these techniques reduce the number of tokens that create the cache in the first place; that is the gap this paper addresses for Arabic specifically.

Arabic is a morphologically rich, templatic language, and standard subword tokenizers, typically built with byte-pair encoding (BPE) [7] and trained predominantly on English-dominated corpora, fragment Arabic text substantially more than English text of equivalent meaning; BPE's merge statistics are learned from the training corpus's character-pair frequencies, so a corpus skewed toward English yields a vocabulary that represents common English morphemes as single tokens far more often than it does Arabic's templatic root-and-pattern morphology. For a fixed context budget this means an Arabic RAG system retrieves less semantic content than an English one, and for a fixed amount of retrieved content it imposes a larger KV-cache footprint. Prompt-compression methods such as LLMingua [8], its long-context variant LongLLMingua [9], Selective Context [10], RECOMP [11], learned soft-prompt compressors such as AutoCompressor [12] and gisting [13] address context size generally but are evaluated largely on English text, without Arabic tokenization disparity or vLLM's KV-cache mechanics as first-class design constraints. This context-size problem is compounded by evidence that LLMs do not use long contexts uniformly: performance degrades when relevant information is buried in the middle of a long context rather than at its start or end [14], which motivates pruning irrelevant content rather than simply retrieving more of it.

This tokenization gap is a practical deployment barrier, not only an academic inefficiency, in settings where GPU memory or inference budget is already the binding constraint. Commercial deployments billed per token pay a direct cost penalty for Arabic relative to English at the same query volume, consistent with the per-language cost inequity in commercial API pricing that Ahia et al. document [15]; the same inflation raises the GPU memory a self-hosted deployment must reserve per concurrent Arabic-language request, which is the more binding constraint for the low-resource, on-premise, single-GPU deployments common outside large commercial labs. This is not a hypothetical constraint: a 2025 review of the Arabic large language model landscape identifies high computational demand relative to regional business resources as a specific, named barrier bridging Arabic NLP research to commercial deployment, alongside limited regional compute infrastructure and cross-border collaboration [16], and a separate compact Arabic language model effort motivates its own design explicitly around bridging the resource demands of large-scale models and the practical constraints of real-world Arabic-language deployment [17]. Reducing the number of tokens a retrieved passage occupies before it

reaches the KV cache, this paper's central mechanism, is one lever available in exactly this setting, alongside the model-side memory techniques discussed as complementary, unevaluated-in-combination directions in Section 5, item 4a, which act on a different axis of the same cost.

We propose a Lightweight Semantic Pruning Middleware (LSPM), a thin layer between retriever and generator that (i) splits retrieved passages into sentences, (ii) scores each sentence's relevance to the query with a cross-encoder [18], and (iii) reconstructs a compact, order-preserving context from the highest-scoring sentences, at a ratio that can be fixed or computed dynamically from vLLM's live /metrics endpoint.

This paper is principally an architecture and validation study of context selection for Arabic RAG: the tokenization measurement, the naive-truncation baseline comparison, the end-to-end retrieval check, and the human evaluation are its central evidence. It additionally includes a GPU benchmark against a self-hosted vLLM server, now executed under the corrected protocol: seven context conditions crossed with five concurrency levels, each cell repeated three times in randomized order with pruning moved out of the load generator, for 105 runs and 191,652 requests with zero failures. That benchmark, scoped to generation cost only (Section 3.1 quantifies the pruning step's own cross-encoder cost separately), finds that semantic pruning imposes no measurable throughput or time-to-first-token penalty relative to unpruned serving at any concurrency level tested; an 8.5-11.9% throughput deficit attributed to LSPM by a preliminary, unrepeated run that scored contexts inside the load generator itself does not survive removal of that load-generator-side scoring cost. It also finds that KV-cache occupancy never rose above 9% of capacity on this hardware, so the benchmark establishes the absence of a serving-cost penalty rather than a demonstrated memory benefit under pressure. A first ablation of the dynamic ratio controller under this same low-pressure regime likewise never reached the 75% occupancy threshold at which the controller would begin adapting, so it validated the controller's plumbing and stability but not its adaptive path; a second, redesigned ablation using a deliberately constrained cache pool subsequently drove occupancy to 99.2% and exercised the controller's full configured ratio range, showing that the adaptive path does engage under real cache pressure. That redesigned run does not, however, show that adapting the ratio improves throughput, tail latency, or answer quality relative to a fixed ratio, because the dynamic and fixed conditions were not matched on average retained context. Both open items identified here have since been closed with new measurements, reported in Sections 4.11 and 4.12: a live, concurrent-load benchmark that scores and prunes context inside the online request path rather than reading it from a precomputed lookup, and a controller-versus-matched-fixed-ratio comparison run under the same constrained-cache-pool pressure as the redesigned ablation. Neither result is favorable enough to report as an unqualified improvement: the true end-to-end benchmark finds a substantial, concurrency-dependent throughput and completion-rate penalty relative to raw serving once the pruning step's own cost is included in the request path, and the controller comparison is a mixed result on serving latency with no answer-quality data collected. The scope and limits of every claim in this paper, measured, analytical, and not-yet-validated alike, are stated explicitly in the sections that follow.

Contributions: (1) A concrete system architecture and open reference implementation for sentence-level, cross-encoder-driven context pruning for Arabic RAG on vLLM-class serving infrastructure, including an implemented dynamic compression-ratio controller that couples pruning aggressiveness to real-time GPU KV-cache occupancy. This controller is a working engineering mechanism, not yet a validated improvement: a redesigned ablation confirms it engages its full adaptive range under real cache pressure, and a further like-for-like comparison against a matched fixed ratio under the same pressure (Section 4.12) finds a mixed result on serving latency with no answer-quality data collected, so

adapting the ratio is not shown to outperform a well-chosen fixed one. (2) A quantitative measurement of Modern Standard Arabic-English tokenization disparity on a 30-pair hand-verified parallel corpus (bootstrap $p < 0.0001$ for both of two tokenizers tested). This is not a novel phenomenon: tokenizer fertility gaps between English and morphologically richer or less-represented languages are established in the literature, from Rust et al.'s demonstration that a language's tokenizer quality, not only its pretraining data volume, drives multilingual model performance [19], to Petrov et al.'s systematic measurement of encoding-length disparity across languages [20] and Ahia et al.'s analysis of the resulting per-language cost inequity in commercial API pricing [15]. Our contribution is to confirm that literature on a hand-verified Arabic corpus with the two specific tokenizers used in this study and, more to the point, to carry it into a serving-system setting by connecting the fertility gap to the KV-cache footprint a vLLM-class engine must hold per request, which is a consequence those studies do not measure. (3) A real, paired baseline comparison against naive sentence-count-matched truncation, re-tested at increasing scale up to a 140-question ARCD re-test (980 live generations), resolving the head-to-head comparison in LSPM's favor at the most aggressive ratio ($r = 0.3$) after Holm-Bonferroni correction, while leaving it open at more moderate ratios. (4) A complementary end-to-end retrieval check that removes the answer-guaranteed pool construction, with a real TF-IDF retriever ($\text{recall}@6 = 86.7\%$), reproducing the same directional advantage. (5) An analytical, explicitly-labeled KV-cache savings projection derived from real per-request prompt-token counts, together with a corrected, repeated GPU throughput/TTFT/KV-cache benchmark against a self-hosted vLLM server (35 cells $\times$ 3 repeats, 105 runs, 191,652 requests, zero failures) that measures the serving-cost consequences of pruning under load, and two dynamic-controller ablations on real, preserved per-request logs, the second of which redesigns the first to reach genuine cache pressure; all are reported with their scope limits stated, including the finding that the main serving benchmark's KV-cache occupancy never approached saturation on the tested hardware, and that the redesigned controller ablation, while confirming the adaptive path does engage at 99.2% occupancy elsewhere, has not shown that adapting improves throughput, latency, or answer quality relative to a matched fixed ratio. A further true end-to-end online benchmark (Section 4.11), scoring and pruning context inside the live request path rather than from a precomputed lookup, closes this paper's most important remaining open item by measuring a substantial, concurrency-dependent throughput and completion-rate penalty for LSPM relative to raw serving, in contrast to the generation-only benchmark's zero-measurable-penalty finding. (6) A blind, two-rater human evaluation (90 answers, correctness and faithfulness, substantial-to-near-perfect inter-rater agreement) that directionally corroborates the automatic-metric result. (7) A token-budget-matched rerun that gives naive truncation the same per-question token budget LSPM actually spent, closing the token-budget confound left open by contribution (3) and reproducing LSPM's advantage at $r = 0.3$. (8) A head-to-head comparison against LLMingua-2, a modern learned prompt compressor, on the identical 140-question evaluation, showing LSPM significantly ahead on token-F1 at every ratio after correction, most convincingly at $r = 0.3$ and 0.5 , with the $r = 0.7$ gap the weakest of the three (its bootstrap interval and rank-based test disagree at the margin; Section 4.8). (9) A generalization check that reruns the full ARCD ground-truth protocol against a second generator, Qwen2.5-7B-Instruct, replicating the central $r = 0.3$ quality result with the same or greater statistical strength. (10) An independent, fresh-seed replication of the central $r = 0.3$ result on 140 newly sampled ARCD questions sharing no question identifiers with the original 140, confirming the finding on data that were not used to set the original sample size or produce the original result.

The remainder of this paper is organized as follows. Section 2 reviews related work in context compression, retrieval, and Arabic natural language processing, and positions LSPM's contribution relative to it. Section 3 describes LSPM's architecture, the experimental setup, and the reproducibility details needed to replicate every reported result. Section 4 reports the quantitative results: tokenization disparity, the ARCD baseline comparison, the end-to-end retrieval check, the human evaluation, the corrected GPU benchmark, and the dynamic-controller ablation (Sections 4.1-4.7), followed by three further validation studies (Sections 4.8-4.10) and two studies run specifically in response to peer review, a true end-to-end online serving benchmark and a controller-versus-matched-fixed-ratio comparison (Sections 4.11-4.12). Section 5 discusses what these results do and do not establish, states the study's limitations explicitly, and outlines the experiments required before a systems-level deployment claim would be justified. Section 6 concludes.

2. Related Work

RAG was introduced by Lewis et al. as a way to combine a parametric generator with retrievable, non-parametric memory for knowledge-intensive NLP tasks [2]; our system is agnostic to the RAG variant used upstream and operates purely on retrieved passages. A broader survey of RAG architectures situates LSPM within the post-retrieval processing stage of the pipeline [21], and several RAG variants make the retrieval-generation loop itself adaptive: REALM jointly pretrains a retriever with the language model [22], kNN-LM instead augments a fixed LM at inference time with a nearest-neighbor datastore [23], Fusion-in-Decoder fuses many retrieved passages inside the decoder rather than concatenating them into a single prompt [24], In-Context RALM shows that simply prepending retrieved documents to the input, without further LM training, already improves language modeling [25], Self-RAG trains the generator to critique and selectively invoke its own retrieval calls [26], Adaptive-RAG routes queries to different retrieval strategies based on estimated question complexity [27], and Self-Ask decomposes a compositional question into retrievable sub-questions before answering [28]; LSPM is compatible with any of these upstream retrieval strategies because it operates only on whatever passages are eventually handed to the generator, and Adaptive-RAG's complexity-conditioned strategy selection is a useful conceptual analogue to LSPM's own dynamic compression-ratio controller (Section 3.1), even though the two operate at different points in the pipeline. Sparse lexical retrieval remains a strong baseline in this literature: the BM25 ranking function [29] predates dense retrieval and is still a common component of hybrid pipelines. Dense retrieval methods such as DPR [30] and ColBERT [31] are complementary to LSPM's retriever-agnostic design. The closest line of work is prompt and context compression: LLMingua uses an auxiliary LM to iteratively remove low-information tokens under a budget controller [8]; Selective Context uses self-information to prune redundant content [10]; RECOMP compresses retrieved documents into extractive or abstractive summaries [11]. None of these three works reports a comparison against a naive sentence-count-matched truncation baseline, a gap our baseline comparison begins to address for the sentence-scoring family specifically, and none targets Arabic tokenization disparity or vLLM's KV-cache mechanics explicitly. LSPM differs by performing sentence-level relevance filtering via a cross-encoder [18], [32] scored directly against the query rather than token-level self-information or learned summarization, and by closing a feedback loop with vLLM's live KV-cache occupancy metric that has no analogue in this prior work.

Cross-encoder rerankers have been a staple of the modern retrieval pipeline since Nogueira and Cho showed a BERT-based reranker could substantially improve passage ranking [18]; Sentence-BERT reformulated this family of models for efficient semantic similarity computation [32]. Multilingual models such as BGE-M3 [33] and multilingual

E5 [34] extend this capability to over 100 languages including Arabic, and LSPM's reference implementation supports a BGE-reranker backend as a pluggable, higher-accuracy option; the specific cross-encoder used to produce every result reported in this paper is cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 (Section 3). Progress on multilingual embedding and retrieval quality is tracked by benchmarks such as MIRACL, which covers 18 languages including Arabic [35], and MTEB, which aggregates embedding-model evaluation across a wide range of tasks and languages [36]; the reference implementation's vector index is built with Chroma, though a GPU-accelerated FAISS index [37] is a common production-scale alternative for the same approximate nearest-neighbor search problem. Arabic NLP has matured substantially with the arrival of large pretrained models specific to the language: AraBERT adapted the BERT pretraining recipe to a large Arabic corpus [38], its generative counterpart AraGPT2 adapted GPT-2-style pretraining to Arabic text generation [39], and AraT5 extended this line of work to a text-to-text encoder-decoder architecture spanning translation, summarization, and question generation [40]; CAMELBERT further showed that matching the variant of Arabic (modern standard, dialectal, or classical) seen during pretraining to the variant seen at fine-tuning time matters more than raw pretraining corpus size [41], and the CAMEL Tools toolkit provides the underlying morphological analysis, tokenization, and dialect-identification utilities used across much of this ecosystem [42].

An alternative to LSPM's post-retrieval pruning approach would be to reduce Arabic's tokenization inflation at its source, by replacing the generator's general-purpose, English-skewed BPE vocabulary with an Arabic-specific vocabulary of the kind AraBERT, AraGPT2, and AraT5 (above) train for themselves, or with a multilingual tokenizer explicitly balanced across scripts rather than dominated by a BPE merge statistic learned from an English-heavy corpus. Such a tokenizer would reduce the constant token-inflation factor this paper measures (1.72-1.79x, Section 4.1) for a fixed amount of retrieved content, whereas LSPM instead reduces the amount of retrieved content that reaches the generator at all; the two act on different axes of the same cost and are complementary rather than competing; a better tokenizer does not remove the case for pruning, since the sentence-relevance argument for preferring content-aware selection over positional truncation (Section 3.2) holds regardless of tokenizer choice, and even a perfectly balanced tokenizer still serves a finite context window. A full generation-quality comparison is not run here because it requires a generator pretrained or fine-tuned with a different tokenizer, not a drop-in substitution on the fixed, off-the-shelf checkpoints (Llama-3.1-8B-Instruct, Qwen2.5-7B-Instruct) used throughout this paper; that controlled comparison, holding the generator's architecture and training data otherwise fixed, remains a direct extension of this paper identified in Section 5. What can be measured without a matching generator is the vocabulary-level effect itself, and we measure it directly: re-tokenizing the same 30-pair parallel corpus used for this paper's headline tokenization measurement (Section 4.1) with three Arabic-specific WordPiece vocabularies (AraBERTv2 [38], CAMELBERT-mix [41], and Arabic-BERT [43]) gives a mean Arabic sentence length of 28.5, 18.7, and 17.8 tokens respectively, against Llama-3.1's 33.2, while the same vocabularies tokenize the parallel English sentences far less efficiently (40.1-48.3 tokens against Llama-3.1's 19.3), since their vocabularies are optimized for Arabic rather than balanced across both languages. The Arabic-side reduction is real and substantial (Arabic-BERT uses 46% fewer Arabic tokens than Llama-3.1's tokenizer on this corpus), directly supporting the claim that an Arabic-optimized vocabulary would shrink the KV-cache footprint of the Arabic side of a RAG prompt considerably; the corresponding English-side inflation is not a concern for the retrieved-context use case this paper targets, where the retrieved content is Arabic, but would matter for any bilingual prompt segment (for example an English system prompt) under the same vocabulary. These are masked-language-model tokenizers,

not generative tokenizers paired with a trained decoder, so this measurement bounds the vocabulary-level opportunity without establishing that a generator using such a vocabulary would preserve answer quality; that remains the open extension described above.

Arabic-specific evaluation has matured alongside these models: ORCA aggregates 60 datasets across seven Arabic NLU task clusters into a single benchmark score [44], ALUE covers Arabic sentence classification, natural language inference, and semantic similarity [45], ArabicMMLU adapts the massive multitask language-understanding format to Arabic [46], and the AceGPT paper [47] additionally introduces ACVA, a benchmark targeting Arabic cultural and value alignment specifically, which is why the same citation covers both AceGPT and ACVA here. Belebele's Arabic variants extend parallel multiple-choice reading comprehension evaluation across 122 language variants [48], and XNLI extends cross-lingual natural language inference evaluation to Arabic among 15 languages [49]. The Arabic Reading Comprehension Dataset (ARCD) [50], a human-authored benchmark in the style of SQuAD [51] with roughly 1,400 question-answer pairs over Arabic Wikipedia articles, remains a standard benchmark for Arabic machine reading comprehension and is the dataset our ground-truth re-test uses. None of this substantial body of Arabic NLP work, to our knowledge, has been connected to the systems-level question of KV-cache-efficient serving, which is the specific contribution of this paper.

On the systems side, vLLM's PagedAttention [4] builds on continuous-batching, iteration-level scheduling [52], chunked-prefill scheduling such as Sarathi-Serve's stall-free batches [53], and cache-management techniques such as StreamingLLM's attention sinks [5] and H2O's heavy-hitter eviction [6] operate orthogonally to LSPM: they manage tokens already in the KV cache, whereas LSPM reduces the number of tokens handed to the engine in the first place, so the two families compose naturally. A separate line of serving-efficiency work reduces the per-token cost of the KV cache itself rather than the token count: FlashAttention computes exact attention with an IO-aware kernel that avoids materializing the full attention matrix [54], and attention-head-sharing designs such as multi-query attention [55] and its generalization, grouped-query attention [56], reduce the number of key-value projections stored per token; all of these are complementary to LSPM's token-count reduction rather than substitutes for it, since they shrink the cost per retained token while LSPM shrinks the number of tokens retained. Our fidelity evaluation relies on three complementary automatic metrics: BLEU measures n-gram precision overlap [57], ROUGE-L uses longest-common-subsequence overlap [58], and BERTScore computes similarity using contextual embeddings rather than exact token overlap, correlating more closely with human judgments of semantic equivalence [59]; RAGAS additionally provides a reference-free framework for scoring RAG-specific qualities such as faithfulness and answer relevance [60]. For the confidence intervals and paired significance tests used throughout this paper, we use the nonparametric bootstrap [61], which avoids distributional assumptions that would be difficult to justify at our sample sizes.

Positioning this work against recent context-refinement and compression systems for RAG is instructive. FILCO [62] trains a context-filtering model that scores and removes irrelevant sentences from retrieved passages using lexical and information-theoretic signals, evaluated on six knowledge-intensive tasks; like LSPM it filters retrieved text before generation, but it is trained end-to-end specifically for filtering rather than reusing an off-the-shelf cross-encoder, and it is evaluated only in English. DSLR [63] is closer still in mechanism: an unsupervised pipeline that decomposes retrieved documents into sentences, re-ranks and filters them by a ranking model's score, and reconstructs a coherent passage, which is architecturally the same split-score-reassemble design LSPM uses; DSLR is evaluated on English open-domain QA and does not consider tokenization disparity, a serving engine's KV-cache cost, or a dynamic compression ratio, each of which is this

paper's specific contribution. XProvidence [64] extends a related zero-cost pruning reranker to over 100 languages via cross-lingual transfer and reports minimal-to-no degradation across four multilingual question-answering benchmarks; it is the closest prior work to LSPM in scope (multilingual context pruning for RAG), but it does not target Arabic's tokenization inflation specifically, does not couple pruning to a live serving-engine signal, and does not report a KV-cache or GPU-serving benchmark. xRAG [65] takes a more aggressive approach, replacing a retrieved passage's text entirely with a single fused embedding token reported to cut inference FLOPs by a factor of 3.53; it achieves a substantially higher compression rate than sentence-level pruning, but it requires training a modality bridge and discards the retrieved text's human-readable form entirely, which is incompatible with an architecture, like LSPM's, that must remain compatible with an unmodified generator and vLLM server.

Positioning statement. Relative to this prior work, LSPM's specific claim to novelty is narrow and stated precisely: it is the first system, to our knowledge, that (i) performs sentence-level, cross-encoder-driven context pruning specifically evaluated on Arabic RAG, where tokenization inflation makes the KV-cache cost of retrieved context measurably worse than for English, and (ii) couples pruning aggressiveness to a live serving-engine signal (vLLM's KV-cache occupancy) rather than a static, offline-computed budget. DSLR is the closest prior system on claim (i)'s mechanism specifically: both decompose retrieved text into sentences, score them against the query, and reassemble a pruned passage, but DSLR is evaluated only on English open-domain QA with an unsupervised ranker and considers neither a tokenization-disparity motivation, a serving engine's KV-cache cost, nor a live compression-ratio signal, which is exactly the (i)/(ii) gap this paper's novelty claim occupies. LSPM has now been compared directly against a genuine modern learned compressor, LLMLingua-2, and significantly outperforms it on token-F1 at every ratio tested, most convincingly at $r = 0.3$ and 0.5 (Section 4.8); it has not been compared against LLMLingua's original iterative-removal variant, Selective Context, DAC, ACC-RAG, or RECOMP, and that broader comparison against the wider compression-baseline field remains the most important open validation step (Section 5, item 1). What this paper does establish is that, within its paired comparisons against both naive truncation and LLMLingua-2, sentence-level semantic selection is a better default for Arabic RAG at aggressive compression, and it reports that result, and only that result, without overstating it into a claim of superiority over every compression method in the literature.

To state this decomposition as plainly as possible: the split-score-reassemble pruning mechanism itself is not algorithmically new and is not claimed as such anywhere in this paper; it is architecturally the same design DSLR already uses. What this paper adds falls into two other categories. The engineering integration is the dynamic controller that couples the compression ratio to vLLM's live KV-cache occupancy signal at request time, a serving-system coupling neither DSLR nor any other prior sentence-level pruning system implements. The empirical validation is everything in Section 4: the tokenization-disparity measurement connecting Arabic morphology to KV-cache cost specifically, the paired comparisons against naive truncation and against LLMLingua-2 on real Arabic RAG data, the corrected GPU serving benchmark, and the two controller ablations. A reader should therefore not take this paper as proposing a new pruning algorithm; it proposes an existing pruning mechanism's serving-aware integration, and validates both the mechanism and that integration on Arabic RAG specifically, which is where this paper's contribution actually lies.

3. Materials and Methods

3.1. System Design

Figure 1 shows LSPM's pipeline. A user query is embedded and matched against a vector database (Chroma) to retrieve a candidate passage pool. LSPM splits each retrieved passage into sentences using a generic, dependency-free regular-expression splitter that breaks on whitespace following a sentence-final punctuation mark (the Latin full stop and exclamation mark and the Arabic question mark U+061F, together with the ASCII question mark), with the Arabic comma U+060C deliberately excluded as a non-terminator and a bare-period fallback applied when no terminator is found; no morphological analyser is used, and in particular CAMEL Tools [42] is cited in Section 2 as part of the Arabic NLP ecosystem but is not a dependency of the released implementation. Because the splitter defines the unit over which the compression ratio r is applied, the same splitter is used for both LSPM and the naive-truncation baseline, so the two arms operate on an identical sentence segmentation.

It then pairs every extracted sentence with the query and scores it with a cross-encoder; the reference implementation supports a fast MiniLM-family backend for interactive use or a BGE-reranker-v2-m3 backend for maximal accuracy as a configurable choice, and cross-encoder/mmarco-mMiniLMv2-L12-H384-v1 specifically is the model used to produce every result reported in this paper (Section 3.3); none of this paper's quality or serving-cost results were reproduced on the BGE-reranker-v2-m3 backend or any other cross-encoder at this paper's full scale; a targeted 20-question check against that second backend is reported in Section 5, item (4b), so backend-to-backend variance is measured on a small subset rather than not measured at all, and should not be assumed to be small beyond that subset. This scoring step has a measured mean latency of 506.1 ms, single-request and CPU-only, over the same six-document candidate pool used in this paper's main baseline comparison (420 real measurements, Section 4.3); Section 4.11 reports the GPU-accelerated figure this same step produces when it runs on the same GPU as the vLLM server, inside a live concurrent-load benchmark. Every throughput/latency comparison in this paper that is described as "generation-only" (Sections 4.6, 5, and 6) reads contexts from a precomputed lookup and therefore measures the vLLM server's generation cost alone, excluding this 506.1 ms CPU cross-encoder cost; it is not an end-to-end online-serving measurement, and should not be read as one. The highest-scoring sentences are retained, in their original order, up to a target compression ratio r (the fraction of sentences kept), and reassembled into a compact context that is handed to the generator running behind a vLLM server.

An optional dynamic controller can adjust r at request time from vLLM's live /metrics KV-cache occupancy: as occupancy rises toward a configured threshold, r is tightened; as it falls, r is relaxed. This controller is an implementation feature of the released system and is described here as a design contribution. It is exercised against live, time-varying traffic in two ablations (Section 4.7, fixed versus dynamic r under a ramping concurrency profile). The first confirms that the controller runs correctly and stably against a real server but did not generate enough KV-cache pressure to cross its high-load threshold, so it validated the controller's plumbing and stability without exercising its adaptive path. A second, redesigned ablation, using a deliberately constrained cache pool, subsequently drove occupancy to 99.2% and exercised the controller's full configured ratio range, confirming that the adaptive path does engage under real cache pressure. Neither ablation shows that adapting the ratio improves throughput, tail latency, or answer quality relative to a fixed ratio matched on average retained context; that comparison remains open and is identified explicitly as required future work (Section 5).

Worked example. The following short example fixes every term used above to a concrete transformation, using one query and a four-sentence retrieved pool. Consider the query "متى تأسست جامعة بغداد؟" ("When was the University of Baghdad founded?")

and a retrieved passage pool that splits into four sentences S1-S4 (shown below), given here in their original document order. Deliberately, the answer-bearing sentence is neither first in the document nor the sentence a positional heuristic would favor: S1 mentions Baghdad generally, S2 is an irrelevant population statistic, S3 states the founding year (the answer), and S4 lists unrelated faculties. The cross-encoder scores each sentence against the query independently, giving $S1 = 0.65$, $S2 = 0.20$, $S3 = 0.93$, and $S4 = 0.08$; note that the highest-scoring sentence, S3, is neither first nor second in the document.

At a compression ratio $r = 0.5$ and $N = 4$ sentences, the number of sentences retained is $k = \max(1, \text{round}(r \times N)) = \max(1, \text{round}(2.0)) = 2$. Ranking S1-S4 by score in descending order gives the sequence [S3 (0.93), S1 (0.65), S2 (0.20), S4 (0.08)]; the top $k = 2$ by this ranking are {S3, S1}. Critically, LSPM does not concatenate the kept sentences in score order (which would read S3 then S1): it restores their original document order before reassembly, so the pruned context is S1 followed by S3, exactly as they appeared in the source passage, and S2 and S4 are dropped. The generator then receives this two-sentence pruned context together with the query and answers "1957", the year contained in S3. Naive length-matched truncation, by contrast, keeps sentences by position rather than score: at the same $r = 0.5$ it keeps the first $k = 2$ sentences by position, {S1, S2}, which does not include S3 and therefore loses the year entirely -- this is the concrete case in which LSPM's content-conditioned cut outperforms a positional cut, not merely an illustration of the mechanism in the abstract. The sentence pool below also shows the outcome of this same ranking at two further ratios for comparison: at $r = 0.3$, $k = \max(1, \text{round}(1.2)) = 1$ and only S3 survives under LSPM, again the correct answer-bearing sentence, whereas naive truncation at $r = 0.3$ keeps only S1 by position and again loses the answer; at $r = 0.7$, $k = \max(1, \text{round}(2.8)) = 3$ and S1-S3 survive under both methods (S4 alone is dropped), so the two methods coincide at this least aggressive ratio.

This illustrative sentence pool (not one of the paper's numbered results tables) makes the ranking and cutoff concrete:

Sentence	Score	Kept@ $r=0.3$	Kept@ $r=0.5$	Kept@ $r=0.7$	Content (English gloss)
		($k=1$)	($k=2$)	($k=3$)	
S1	0.65	--	yes	yes	"Baghdad is Iraq's capital, located on the banks of the Tigris River."
S2	0.20	--	--	yes	"Baghdad's population exceeds seven million."
S3	0.93	yes	yes	yes	"...founded in 1957, becoming one of the most prominent Iraqi universities."
S4	0.08	--	--	--	"The university includes several colleges, including engineering and medicine."

This example also illustrates why LSPM's advantage over naive truncation should be expected to grow as r shrinks (the mechanistic argument formalized in Section 3.2): a positional cut keeps whichever sentences happen to occupy the retained positions regardless of content, so it loses the answer at both $r = 0.3$ and $r = 0.5$ above, precisely because the answer-bearing sentence (S3) does not happen to occupy an early position in this retrieved pool; LSPM's cut is conditioned instead on the cross-encoder's relevance estimate, so an answer-bearing sentence anywhere in the pool, not only near the start, survives an aggressive cut as long as it scores highly enough, and the survivors are still returned to the generator in their original document order rather than in score order.

Algorithm 1: LSPM pruning. The pseudocode below states the pruning step precisely. Terms: q is the user query (a string); $D = (d_1, \dots, d_m)$ is the ordered list of retrieved documents in retrieval-rank order; $\text{Split}(d)$ returns d 's sentences in their original

within-document order (Section 3.1's whitespace/punctuation splitter); r in $(0, 1]$ is the compression ratio (fraction of sentences retained); `min_keep` is a floor on the number of sentences kept (1 throughout this paper); `CrossEncoder(q, s)` returns a real-valued relevance score for sentence s against query q (higher is more relevant); `round(.)` is round-half-to-even; a stable sort is one that preserves the original relative order of elements with equal keys.

Algorithm 1. LSPM-Prune($q, D, r, \text{min_keep} = 1$): sentence-level semantic pruning with order-preserving reconstruction.

Input: query q ; ordered documents $D = (d_1, \dots, d_m)$; ratio r in $(0, 1]$; `min_keep`

Output: pruned context string

```

1: S <- empty ordered list           // every sentence, original order
2: for each document d in D (retrieval order):
3:   for each sentence s in Split(d) (document order):
4:     append s to S
5: N <- |S|
6: for i <- 1 .. N: score[i] <- CrossEncoder(q, S[i])
7: k <- max(min_keep, round(r * N)); k <- min(k, N)
8: rank <- indices 1..N sorted by score[.] descending, stable
                               // ties keep earlier original index first
9: keep <- first k indices of rank   // top-k by score
10: S_pruned <- [ S[i] for i in 1..N if i in keep ] // restore ORIGINAL order
11: context <- Join(S_pruned, " ")
12: return context

```

Algorithm 2: dynamic ratio controller. The optional controller of Section 3.1 maps live KV-cache occupancy to a ratio using the same r as Algorithm 1. Terms: `usage` in $[0, 1]$ is the current GPU KV-cache occupancy fraction, read from vLLM's `/metrics` endpoint (`gpu_cache_usage_perc`); `r_min` and `r_max` bound the returned ratio (0.20 and 0.80 throughout this paper, Section 3.5.3); `theta_low` and `theta_high` are the low- and high-load occupancy thresholds (0.25 and 0.75) between which the ratio is linearly interpolated; `clamp(x, a, b)` restricts x to $[a, b]$.

Algorithm 2. DynamicRatio($\text{usage}; r_{\text{min}}, r_{\text{max}}, \text{theta}_{\text{low}}, \text{theta}_{\text{high}}$): occupancy-to-ratio mapping.

Input: `usage` in $[0, 1]$; `r_min`, `r_max`; `theta_low`, `theta_high` (`theta_low < theta_high`)

Output: r in $[r_{\text{min}}, r_{\text{max}}]$

```

1: if usage >= theta_high: return r_min   // heavy load: prune aggressively
2: if usage <= theta_low:  return r_max   // light load: keep richer context
3: frac <- (usage - theta_low) / (theta_high - theta_low)
4: r <- r_max - frac * (r_max - r_min)    // linear interpolation
5: return clamp(r, r_min, r_max)

```

3.2. Why Semantic Pruning Should Preserve Answer Quality Better Than Truncation

The mechanistic rationale for expecting LSPM to outperform positional truncation, particularly at aggressive compression, follows directly from where answer-bearing content tends to sit in a retrieved passage pool relative to where a fixed-position cut falls. Naive truncation keeps the first $r \cdot N$ sentences of the concatenated retrieved pool regardless of their content; if the answer-bearing sentence for a given question falls after that cut point, which becomes increasingly likely as r shrinks and the retrieval pool mixes gold and distractor passages, truncation removes it outright and no downstream generation strategy can recover it. LSPM instead ranks every sentence by a cross-encoder's estimate of its relevance to the specific query, so the retention decision is conditioned on content rather than position: an answer-bearing sentence that would fall outside a positional cut can still be retained if it scores above the compression threshold. This predicts a specific, testable pattern, namely that LSPM's advantage over truncation should widen as r shrinks (more content is discarded, so position becomes an increasingly poor proxy for relevance) and should be concentrated on questions where the gold sentence is not near the start of the retrieved pool. The result reported in Section 4 is consistent with the first half of this prediction: the significant head-to-head advantage for LSPM is concentrated at the most aggressive ratio tested ($r = 0.3$) and is not significant after correction at $r = 0.5$ or $r = 0.7$, exactly where truncation's positional cut is least likely to exclude the answer in the first place. We did not directly test the second half of this prediction, position of the gold sentence within the retrieved pool, as an explicit moderating variable in this study; doing so, and reporting whether LSPM's advantage is in fact concentrated on late-positioned gold sentences, is identified as a direct, low-cost follow-up analysis on the existing ARCD re-test data (Section 5).

3.3. Experimental Setup

Tokenization disparity: a hand-verified, 30-pair Modern Standard Arabic (MSA)-English parallel corpus was tokenized with two contemporary tokenizers (Qwen2.5-7B-Instruct [66], Llama-3.1-8B-Instruct [67]), and a one-sample bootstrap test (10,000 resamples) tested the mean ratio against parity. Two related quantities are reported for this disparity in this paper and should not be conflated: the mean per-pair token ratio (each pair's Arabic token count divided by its English token count, then averaged across the 30 pairs, reported in Section 4.1 and Table 1 as 1.79x for Qwen2.5 and 1.72x for Llama-3.1) and an aggregate ratio (total Arabic tokens across all 30 pairs divided by total English tokens across all 30 pairs, approximately 1.76x-1.69x across the two tokenizers). The two are related but numerically distinct aggregation procedures. To state plainly which of them appears where: the Abstract quotes the per-pair figures (1.79x and 1.72x), and the Conclusions quote the aggregate figures (1.69x-1.76x). Both are computed on the same underlying 30-pair corpus and both reject parity at $p < 0.0001$, but a per-pair mean and a ratio-of-sums are not required to be numerically identical, and this paper reports both explicitly rather than treating them as interchangeable.

Fidelity pilot ($n = 8$): a small pilot compared LSPM against raw (unpruned) context and naive truncation at r in $\{0.3, 0.5, 0.7\}$ using BLEU, ROUGE-L, and BERTScore-F1 against the unpruned answer as a reference.

3.4. ARCD Ground-Truth Re-Test

ARCD ground-truth re-test ($n = 140$): to address the small pilot's scale and its scoring against the unpruned answer rather than a real gold answer, we sampled 140 questions from ARCD's validation split [50] (seed = 42), built a six-document pool per question guaranteeing the gold paragraph is present (recall@6 = 100% by design) plus five distractors from unrelated articles, and generated seven answers per question against the live Llama-3.1-8B-Instruct endpoint: raw (unpruned), LSPM at r in $\{0.3, 0.5, 0.7\}$, and naive

truncation at the same three ratios (980 live generations total). This candidate-pool construction is deliberate and should be read precisely for what it tests: the gold paragraph's guaranteed presence and the five unrelated-article distractors remove retrieval failure and topical-similarity confusion as variables, isolating post-retrieval sentence selection as the object of study. This is easier than a realistic retrieval setting, where the gold passage may not be retrieved at all and surviving distractors are often topically similar rather than drawn from unrelated articles; the claim this design supports is accordingly scoped to post-retrieval selection quality under controlled candidate-pool construction, not to end-to-end RAG accuracy under realistic full-corpus retrieval with hard, topically similar negatives. Section 4.4's end-to-end check relaxes the retrieval-failure half of this by allowing a real TF-IDF retriever to miss the gold paragraph outright, but its 234-paragraph corpus and single lexical retriever do not yet supply the harder, topically similar negatives that a larger corpus or a BM25/dense/hybrid retriever would; that stronger evaluation remains open work (Section 5). Answers are scored against ARCD's gold answer string(s) with Arabic-normalized Exact Match (EM) and token-F1. We report paired differences with bootstrap 95% CIs, Wilcoxon signed-rank tests, Holm-Bonferroni correction across a nine-test family, and a two one-sided equivalence test (TOST, margin ± 0.05 F1) against the raw answer, following the two one-sided tests procedure originally developed for bioequivalence testing [68] and its practical primer for behavioral and NLP-adjacent research [69], which we use here to distinguish a genuine null result (no evidence of a difference) from a positive finding that LSPM's pruned answer is statistically equivalent, within a pre-specified margin, to the unpruned answer. Both LSPM and naive truncation are matched by the number of sentences retained (the compression ratio r is applied identically to both, as a fraction of retained sentences); we did not additionally match the two methods by the number of retained input tokens, and because the sentences each method selects can differ substantially in length, an equal sentence count does not guarantee an equal token budget or equal KV-cache cost between the two conditions. This is disclosed explicitly as a limitation on the efficiency interpretation of the main comparison (Section 5). Two remedies close this gap, one analytical and one experimental: Section 4.3 reports the actual prompt tokens each method spent, computed offline from the released artifacts, which is a partial, descriptive fix (it shows what token budget the existing sentence-matched runs happened to use, without rerunning anything); Section 3.5.4 describes the fuller, experimental fix, a rerun in which naive truncation is given LSPM's own realized token budget directly rather than merely having it reported after the fact, with the result in Section 4.8. All inferential tests in this paper use a two-sided alpha of 0.05, which is the level at which the Holm-Bonferroni family-wise correction, the 95% bootstrap confidence intervals, and the TOST equivalence tests reported below are all evaluated.

Sample-size provenance and the earlier 40-question pilot. An earlier phase of this project ran the identical ARCD pipeline at $n = 40$ before the $n = 140$ re-test reported here, and the sample-size target for the larger run was derived from that pilot's observed effect size. The pilot's records were not lost, and its relationship to the 140-question sample is not independence: the 40 questions are exactly the first 40 questions of the 140. The sampling script (`build_arcd_eval_set.py`) seeds Python's random module with `SEED = 42` and shuffles the full ARCD validation split before taking the first N distinct-article questions, so raising N from 40 to 140 extends the sample by appending rather than redrawing it. We verified this directly against the released artifacts: exactly 280 of the 980 rows in `arcd_results.jsonl` (40 questions $\times$ 7 conditions) carry question identifiers, and those identifiers are precisely the first 40 entries of `arcd_eval_set.json`. The 40 pilot questions are therefore a strict subset of the 140, not a disjoint pilot, and we state that explicitly rather than leave it to be inferred. Recomputing the pilot from those rows reproduces its reported outcome: at $r = 0.3, 0.5$, and 0.7 the paired LSPM-minus-naive token-F1 differences are

+0.086, +0.083, and +0.084, none significant at $n = 40$ (Wilcoxon $p = 0.171, 0.105, 0.094$), with standardized paired effect sizes (Cohen's d for paired differences) of 0.248, 0.251, and 0.252; a conventional paired-sample power calculation at 80% power and two-sided alpha = 0.05 on those effect sizes returns approximately 126 to 131 questions, which is where $n = 140$ came from. Because that effect size was estimated from data that are themselves part of the confirmatory sample, this is an observed-power calculation, and the resulting sample-size justification is circular: computing power from an effect size observed in one's own data carries no additional information beyond the p -value already obtained and cannot be used to certify that a subsequent test was adequately powered [70]. We therefore withdraw the power calculation as a justification. The $n = 140$ re-test reported below should be read as a pragmatically chosen sample size, not a formally powered one, and because the decision to extend to 140 was taken after inspecting a subset of the same data, the re-test is a data-dependent continuation of the pilot rather than a fully independent confirmatory experiment. The practical consequence for interpretation is stated in Section 5.

Reproducibility details. Generation model: meta-llama/Llama-3.1-8B-Instruct, accessed for the fidelity, baseline, ARCD, and end-to-end experiments via NVIDIA NIM's hosted OpenAI-compatible API (model identifier meta/llama-3.1-8b-instruct, accessed August 2026; NIM resolves this identifier to a pinned checkpoint on its serving side, which we do not independently verify beyond the identifier string), temperature = 0, max 64 tokens (ARCD/end-to-end) or 200 tokens (fidelity pilot). The GPU benchmark and controller ablation described in Sections 4.6 and 4.7 instead self-host the same model (bf16, --max-model-len 8192, served via the NousResearch mirror of the checkpoint to avoid the gated-repository dependency) under vLLM 0.28.0 on a single rented RTX 4090 (48 GiB). The server version and hardware for that run are recorded here from the authors' deployment record; the released per-cell artifacts capture the measurements but do not themselves embed a server version string, which we note rather than imply otherwise. Cross-encoder: cross-encoder/mmarco-mMiniLMv2-L12-H384-v1. Random seeds: SEED = 42 (Python random, ARCD sampling and distractor selection), numpy.random.default_rng(seed=42) (bootstrap resampling), seed = 123 (end-to-end retrieval question sample). Analysis code: Python 3.10, scipy 1.15, numpy 2.2. Complete source, configuration, and launch scripts are released (Data Availability Statement).

Use of generative AI in this study. Large language model assistance (Anthropic Claude) was used for software implementation (the LSPM middleware, evaluation scripts, statistical analysis scripts, ARCD sampling and Arabic EM/F1 scoring code), for literature-search assistance in identifying candidate related work subsequently verified by the authors against primary sources, and for drafting assistance on this manuscript under the authors' direction and review. All experimental results reported in this paper were produced by executing the released code against live infrastructure; no results were generated, adjusted, or selectively reported without corresponding code execution, and non-significant and inconclusive findings are reported in full rather than omitted. The authors take full responsibility for the accuracy of all claims, citations, and reported results in this paper.

3.5. Protocols for the Extended Validation Studies (Sections 4.5-4.12)

Beyond the core ARCD re-test above, this paper reports eight further validation studies. To keep design decisions in Methods and outcomes in Results, this subsection specifies what each study did before Section 4 reports what each study found.

3.5.1. Human evaluation protocol (results in Section 4.5). Two blind evaluators rated 90 answers drawn from the ARCD re-test generations, balanced as 30 answers per condition across raw (unpruned), LSPM at $r = 0.3$, and naive truncation at $r = 0.3$, with condition labels held in a private codebook so that raters saw only the question, the source

context, the gold answer string(s), and the candidate answer. Correctness was rated on a three-point scale (0 = incorrect, 1 = partially correct, 2 = fully correct) and faithfulness on a binary scale (0 = not grounded in the provided context, 1 = grounded). Following the released protocol (HUMAN_EVAL_PROTOCOL.md), both raters are volunteer bilingual annotators who are not the authors and who took no part in building or evaluating the system, and both were kept unaware of the study's hypothesis; each rated all 90 items independently and without conferring, so neither rater saw the other's ratings. Disagreements were left unresolved by design rather than adjudicated.

3.5.2. GPU benchmark protocol (results in Section 4.6). This benchmark measures what pruning costs, or saves, in real serving terms: request throughput, time-to-first-token, and KV-cache occupancy as a percentage of capacity, under controlled concurrent load. It is not a test of the analytical KV-cache projection of Section 4.3; that projection is denominated in bytes per request, whereas this benchmark reports occupancy as a fraction of the engine's total cache capacity. A Locust-based harness issued concurrent chat-completion requests against a self-hosted vLLM server (Llama-3.1-8B-Instruct, bf16, --max-model-len 8192) on a single rented RTX 4090 (48 GiB). The design is a seven-by-five grid: seven context conditions (raw, i.e. unpruned; LSPM at r in {0.3, 0.5, 0.7}; and naive sentence-count-matched truncation at the same three ratios) crossed with five concurrency levels (1, 10, 25, 50, and 100 concurrent clients), giving 35 cells, each repeated three times, for 105 independent 90-second load-generation runs. Two confounds present in an earlier, exploratory run (now superseded; disposition in the Data Availability Statement) are removed by construction here: pruned contexts are precomputed offline (precompute_contexts.py) and looked up by key at request time, so no cross-encoder scoring runs inside the load generator for any method at any concurrency level; and all 105 cells are pooled and shuffled under a fixed seed before execution (run_full_sweep.py --repeats 3), so elapsed run time and warm-up state are decorrelated from method identity rather than confounded with it. One control this design calls for was not implemented in the released harness: the server is not restarted between blocks but runs continuously across the shuffled cell order, which randomizes rather than eliminates server-state drift; we disclose this gap rather than describe a control we did not exercise. Every cell is the mean of its three repeats with a 95% confidence interval computed from the t-distribution across repeats.

3.5.3. Dynamic-controller ablation protocol, original and redesigned (results in Section 4.7). The dynamic compression-ratio controller of Section 3.1 was first exercised in a ramping-load ablation against the same self-hosted vLLM server: 60 s per step at 4, 8, 32, 32, 8, and 4 concurrent workers, six minutes per condition, run once with the ratio held constant at $r = 0.5$ and once with the controller re-polling vLLM's live /metrics endpoint before every request; full per-request logs (ratio used, KV-cache occupancy read at request time, TTFT, total latency) were preserved and released as results/controller_ablation. We identified two reasons this original design could under-stress the controller and corrected both for a redesigned run, disclosing the change rather than silently rerunning until a preferred result appeared: first, the original ablation used a small synthetic corpus (~240 tokens per context) rather than the realistic contexts this paper's own workload produces (mean 1,485.6 prompt tokens per request, Section 4.3); second, it ran against a server started at vLLM's default cache utilization (--gpu-memory-utilization ~0.9 on the 48 GiB card used throughout this paper), leaving a KV-cache pool large enough that even 32 concurrent requests barely dented it. The redesigned ablation (v2, results/controller_ablation_v2) uses the real 140-question ARCD context pool in place of the synthetic corpus, a higher concurrency ramp (six 90 s steps at 10, 30, 80, 80, 30, and 10 concurrent workers per condition), and a server deliberately started at --gpu-memory-utilization 0.3, a smaller cache pool chosen specifically to make the cache-pressure threshold

reachable on a single rented GPU without requiring an unrealistic request volume. This is a disclosed methodological choice made to observe the controller's adaptive path at all, not a claim about a recommended production cache setting, and the two ablations are consequently not run under the same server configuration; only the redesigned run should be read as evidence about adaptive behaviour.

3.5.4. Token-budget-matched rerun and LLMingua-2 baseline protocols (results in Section 4.8). Section 4.3 identified the paper's single most important open methodological gap: LSPM and naive truncation were matched by retained sentence count, not retained token count, so the reported F1 advantage was not, by construction, a comparison at equal token budget. To close that gap directly, a new condition, `naive_tokbudget`, is given the same per-question token budget LSPM actually spent (computed per question from the offline replay of Section 4.3) and greedily retains whole sentences up to that budget without exceeding it, so it is matched to LSPM's realized cost rather than to its sentence count. Separately, to compare against a contemporary learned compressor, LLMingua-2 [71] was added as a third context-construction method, run over the same retrieved document pool at the same three ratios, with its own compression budget verified against the same origin context.

3.5.5. Qwen2.5 generalization-check protocol (results in Section 4.9). Every fidelity and quality result in Sections 4.2-4.7 uses a single instruction-tuned generator, Llama-3.1-8B-Instruct, leaving open whether the reported ranking of methods held for a different generator family (Section 5, item 5). We reran the ARCD ground-truth protocol of Section 4.3 in full against Qwen2.5-7B-Instruct [66], self-hosted under the same vLLM configuration used for the corrected GPU benchmark, on the identical 140 questions, same seven conditions, same cross-encoder, same Arabic-normalized scorer (`score_arcd.py`, see the methodological note in Section 4.8), producing 980 new generations.

3.5.6. Fresh-seed disjoint replication protocol (results in Section 4.10). Section 3.4 and Section 5 both note a specific limitation of the $n = 140$ ARCD re-test: its sample size was derived from an internal 40-question pilot that is a strict subset of the same 140 questions, so the decision to extend to $n = 140$ was taken after inspecting a subset of the same data, making the reported result a data-dependent continuation of that pilot rather than an independently confirmed finding. To close that gap directly rather than merely disclose it, we ran a separate, independent replication. A new script (`build_arcd_eval_set_replication.py`) sampled 140 ARCD validation questions with a different random seed (2026, versus the original SEED = 42) and excluded every question identifier already present in the original 140-question set before sampling; a programmatic check confirmed zero overlap between the two id sets. These 140 fresh questions were evaluated under the identical protocol used in Section 4.3, restricted to the three conditions the central result concerns: raw (unpruned), LSPM at $r = 0.3$, and naive truncation at $r = 0.3$, against the same generator (Llama-3.1-8B-Instruct, the same NVIDIA NIM endpoint and prompt template), the same cross-encoder, and the same Arabic-normalized scorer (`score_arcd.py`, Section 4.8), producing 420 new live generations (140 questions $\times$ 3 conditions). The generator, retrieval mechanism, pruner, and scorer were held fixed deliberately: substituting a different generator family here would constitute a further generalization check of the kind already reported in Section 4.9, not a confirmation of the original finding.

3.5.7. True end-to-end online serving benchmark protocol (results in Section 4.11). Section 4.6's GPU benchmark and every "generation-only" comparison in this paper (Sections 4.6, 5, and 6) read a precomputed, already-pruned context from a lookup table before generation, so none of them include the cross-encoder scoring cost quantified separately in Section 3.1. To close this gap, we ran a new script (`benchmark/run_online_e2e_benchmark.py`) against a live, self-hosted vLLM server (a

single rented RTX 4090 with 48 GiB of VRAM; Section 4.11 reports the full hardware disclosure) that performs retrieval, cross-encoder scoring, and pruning inside the request path itself, immediately before each generation call, rather than reading an already-pruned context from a lookup. Three context conditions were tested: raw (unpruned), LSPM at a fixed ratio, and naive length-matched truncation at the same ratio, crossed with four concurrency levels (1, 5, 10, and 20 concurrent requests) and three compression ratios ($r = 0.3, 0.5, 0.7$), for 60 requests per (condition, concurrency, ratio) cell. For every cell we recorded the pruning step's own latency separately from time-to-first-token (TTFT), total request latency, and achieved throughput (requests/s and tokens/s), so the cost of scoring and pruning inside the live request path is visible rather than hidden inside a generation-only figure.

3.5.8. Controller-versus-matched-fixed-ratio comparison protocol (results in Section 4.12). Section 4.7's redesigned ablation established that the dynamic ratio controller's adaptive path engages under real cache pressure (occupancy reaching 99.2%) but, because the dynamic and fixed arms were not matched on average compression ratio, could not show whether adapting the ratio outperforms a fixed one. To close that comparison directly, we ran a new script (scripts/run_controller_vs_matched_fixed.py) under the same --gpu-memory-utilization 0.3 constrained cache pool as the redesigned ablation of Section 4.7, running two conditions back to back on the same RTX 4090 48 GiB card: "dynamic", in which the controller adjusts its ratio in response to live KV-cache occupancy exactly as in Section 4.7, and "matched_fixed", in which the ratio is held constant at the dynamic condition's own realized mean ratio for the run, so the two arms retain the same average compression ratio (a proxy for, but not identical to, an average retained-token match; Section 4.12). For every request in both conditions we logged the ratio in effect, the KV-cache occupancy at request time, TTFT, and total latency; the run was executed with the script's --skip-quality flag, so this comparison covers serving latency only and does not include an answer-quality comparison between the two conditions.

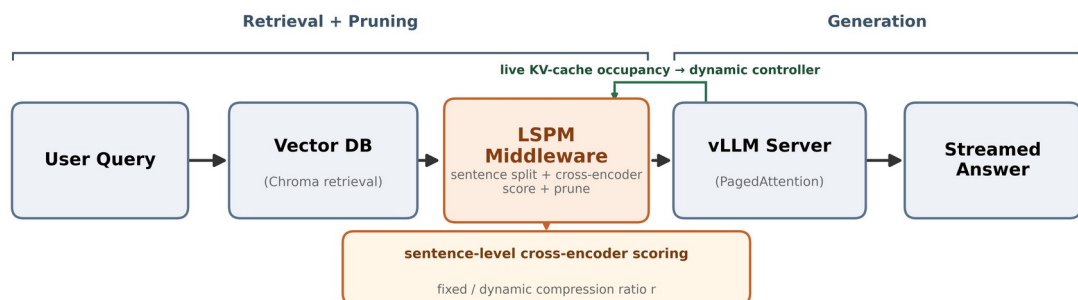


Figure 1. LSPM pipeline: retrieval, sentence-level cross-encoder scoring and pruning, generation, and the dynamic controller's feedback link to vLLM's live KV-cache occupancy metric.

4. Results

4.1. Tokenization Disparity

Table 1 shows a mean per-pair ratio of 1.793 (Qwen2.5, 95% CI 1.691-1.895) and 1.717 (Llama-3.1, 95% CI 1.613-1.820); a bootstrap test rejects parity for both tokenizers at $p < 0.0001$, and a sign test finds 29/30 pairs above parity for both tokenizers. The aggregate ratio (total Arabic tokens / total English tokens across all 30 pairs, distinct from the per-pair mean defined above in Section 3.3) is 1.76x for Qwen2.5 and 1.69x for Llama-3.1. The two aggregation procedures are quoted in different places in this paper and we identify them explicitly to prevent conflation: the Abstract quotes the per-pair means (1.79x and 1.72x), while the Conclusions quote the aggregate ratios (1.69x-1.76x). All 30 pairs are

Modern Standard Arabic, so this measurement characterizes MSA specifically and not Arabic dialects (Section 5). The direction and rough magnitude of this gap are consistent with the cross-linguistic tokenizer-fertility literature [19], [20], [15]; what is added here is the specific pairing of tokenizers, corpus, and the downstream serving-cost consequence developed below.

Table 1. Arabic:English token-count ratio across two tokenizers on 30 hand-verified sentence pairs. Mean per-pair ratio is the per-pair Arabic/English token ratio averaged across pairs; aggregate ratio is total Arabic tokens divided by total English tokens across all 30 pairs.

Tokenizer	Mean per-pair ratio	95% CI	Aggregate ratio	Bootstrap p
Qwen2.5-7B-Instruct	1.793	1.691-1.895	1.76x	< 0.0001
Llama-3.1-8B-Instruct	1.717	1.613-1.820	1.69x	< 0.0001

4.2. Fidelity Pilot

On this small, low-redundancy pilot ($n = 8$), fidelity does not collapse even at the most aggressive ratio tested, and LSPM shows no statistically significant fidelity advantage over naive truncation at any ratio (all bootstrap $p > 0.2$), motivating the larger ARCD re-test below. Full per-ratio ROUGE-L and BERTScore-F1 means and the character-reduction figure are reported in Table S1.

4.3. ARCD Ground-Truth Re-Test

ARCD's questions and source paragraphs are drawn from Arabic Wikipedia and are written in Modern Standard Arabic, so every result in this subsection is an MSA result and should be read as scoped to that variety (Section 5). All 140 questions produced a usable generation in all seven conditions ($140 \times 7 = 980$ generations, zero failures, zero empty completions), so every paired comparison below is computed on the full $n = 140$ without exclusions. Table 2 reports per-condition means with 10,000-resample percentile bootstrap 95% confidence intervals for both scored metrics. The unpruned (raw) condition attains mean token-F1 0.666 (95% CI 0.612-0.718) and mean EM 0.293 (95% CI 0.221-0.371). At the most aggressive ratio tested, LSPM reaches F1 0.652 (0.596-0.707) and EM 0.314 (0.236-0.393) against naive truncation's F1 0.537 (0.474-0.600) and EM 0.250 (0.179-0.321); at $r = 0.5$ the corresponding values are F1 0.640 (0.582-0.695) for LSPM versus 0.576 (0.513-0.639) for truncation, and at $r = 0.7$ they are 0.665 (0.610-0.721) versus 0.604 (0.543-0.663).

At $r = 0.3$, LSPM's mean token-F1 exceeds naive truncation's by 0.114, with a paired bootstrap 95% CI on the difference of +0.056 to +0.176 that excludes zero (Wilcoxon signed-rank raw $p = 0.0008$, Holm-Bonferroni-adjusted $p = 0.0067$); the paired EM difference at the same ratio is +0.064 (95% CI +0.007 to +0.121, Wilcoxon raw $p = 0.029$), consistent in direction with the F1 result. LSPM is statistically equivalent to the unpruned (raw) answer under TOST ($n = 140$, margin ± 0.05 F1, observed mean difference -0.014, $p = 0.037$), while naive truncation is significantly worse than raw (mean difference -0.129, 95% CI -0.185 to -0.073, raw $p = 0.00008$, adjusted $p = 0.0007$). At $r = 0.5$ and $r = 0.7$, the LSPM-vs-naive F1 differences are +0.064 (95% CI +0.004 to +0.124) and +0.061 (95% CI +0.012 to +0.112) and are nominally significant before correction (raw $p = 0.031$ and $p = 0.022$) but do not survive it (adjusted $p = 0.126$ and $p = 0.115$, respectively); the corresponding EM differences (+0.007 and +0.014) are not significant even uncorrected. This pattern is consistent with the mechanistic account in Section 3.2 that semantic selection matters most precisely when positional truncation is most likely to discard the answer-bearing sentence, but it also means the significant advantage reported here should not be read as a general superiority claim across all compression ratios (Section 5).

For auditability of the multiplicity correction, the nine-test family is the nine F1 contrasts formed by the three ratios crossed with the three method pairs; the full ordered list of all nine raw Wilcoxon p-values and their Holm-adjusted counterparts is reported in Table S2 so the correction step can be checked directly rather than taken on trust. The TOST equivalence tests reported above are run on the same data but are not members of this nine-test family and are reported uncorrected. We note explicitly that no standardized effect size (for example a rank-biserial correlation for the Wilcoxon tests) was computed by the released analysis pipeline, so the +0.114 figure should be read as an unstandardized mean paired difference with its bootstrap interval, not as an effect size; the absence of standardized effect sizes is recorded as a reporting limitation in Section 5.

Prompt-token counts and the token-budget rival explanation. Because LSPM and naive truncation are matched by retained sentence count rather than retained token count (Section 3.4), a rival explanation for the $r = 0.3$ quality result is available: relevance-ranked sentences might simply be longer than the first sentences of the pool, so LSPM could have won by spending a larger token budget rather than by selecting better content. We tested that explanation directly. The prompts themselves were not stored in the released results file, but the pipeline that produced them is deterministic given the eval set and the cross-encoder, so we replayed it offline for all 980 cells and counted tokens with the generator's own tokenizer (Llama-3.1-8B-Instruct, non-gated NousResearch mirror, identical vocabulary), including the chat template and system prompt. The replay is verified rather than assumed: for all 980 cells the reconstructed context's character count and sentence count match the values the live run recorded at generation time, with zero mismatches. Mean prompt length is 1,485.6 tokens for the raw condition (median 1,423). At $r = 0.3$ LSPM's prompts average 481.9 tokens against naive truncation's 534.8, a paired difference of -52.9 tokens (95% CI -76.9 to -30.1, Wilcoxon $p = 0.000064$), so LSPM used about 10% fewer tokens than the baseline it beat. The same direction holds at the other two ratios: -60.1 tokens at $r = 0.5$ (95% CI -87.4 to -33.5) and -54.0 tokens at $r = 0.7$ (95% CI -77.0 to -31.7). The rival explanation is therefore not supported; if anything the token accounting runs against it, and the +0.114 F1 advantage at $r = 0.3$ was obtained at a strictly smaller prompt-token budget. Two qualifications keep this honest. First, the advantage is not uniform per question: LSPM's prompt is shorter than naive truncation's in 79 of 140 questions at $r = 0.3$ (91 of 140 at $r = 0.5$, 93 of 140 at $r = 0.7$), so the mean is driven by magnitude as well as by direction. Second, this is an observational reconciliation of the existing run, not the token-budget-matched rerun described in Section 5; it removes the specific rival explanation that LSPM won by spending more tokens, but it does not constitute an experiment in which both arms were held to an identical budget by construction.

KV-cache footprint implied by the measured token reduction. These token counts also let the paper's KV-cache projection be computed from measured tokens rather than from a character-based approximation. For a grouped-query-attention decoder the per-token KV-cache footprint is the product of the number of layers, the number of key-value heads, the head dimension, a factor of two for the separate key and value tensors, and the bytes per element. For Llama-3.1-8B-Instruct as published (32 hidden layers, 8 key-value heads, hidden size 4,096 over 32 attention heads giving a head dimension of 128) served in bf16 (2 bytes per element), this is $2 \times 32 \times 8 \times 128 \times 2 = 131,072$ bytes, i.e. exactly 128 KiB of KV cache per token. Multiplying by the per-request token reduction measured above gives, relative to the unpruned prompt, a saving of 125.5 MiB per request at $r = 0.3$ (95% CI 119.9-131.2), 92.3 MiB at $r = 0.5$ (87.6-97.0), and 57.4 MiB at $r = 0.7$ (54.2-60.6). This is substantially larger than a character-based approximation would suggest: converting the $n = 8$ fidelity pilot's mean character reduction to tokens through an assumed characters-per-token ratio would put the saving at only 9.7-19.1 MiB. That conversion is the wrong

instrument for this paper in particular, since the paper's own tokenization finding is that the character-to-token relationship is exactly what behaves irregularly for Arabic; the figures above avoid it entirely by counting tokens directly. The projection remains analytical in the sense that matters: it is arithmetic on measured token counts, not an observed reduction in GPU memory, and the benchmark of Section 4.6 never entered the cache-pressure regime in which such a saving would translate into a serving benefit (Section 5).

Table 2. Per-condition means for the ARCD ground-truth re-test (n = 140 questions per condition, 980 generations total). EM is Arabic-normalized Exact Match and F1 is Arabic-normalized token-F1, both scored against ARCD's gold answer strings. Intervals are 10,000-resample percentile bootstrap 95% confidence intervals (seed = 42).

Condition	Mean EM	EM 95% CI	Mean F1	F1 95% CI
raw (unpruned)	0.293	0.221-0.371	0.666	0.612-0.718
LSPM, r = 0.3	0.314	0.236-0.393	0.652	0.596-0.707
naive, r = 0.3	0.250	0.179-0.321	0.537	0.474-0.600
LSPM, r = 0.5	0.300	0.229-0.379	0.640	0.582-0.695
naive, r = 0.5	0.293	0.221-0.371	0.576	0.513-0.639
LSPM, r = 0.7	0.300	0.229-0.379	0.665	0.610-0.721
naive, r = 0.7	0.286	0.214-0.364	0.604	0.543-0.663

4.4. End-to-End Retrieval Check

This check uses an independent sample of n = 60 ARCD questions (seed = 123), each answered under three conditions (raw, LSPM at r = 0.3, naive truncation at r = 0.3) against the top six documents returned by a real TF-IDF retriever over a 234-paragraph corpus. Retrieval succeeds for 52/60 questions and fails for 8/60 (recall@6 = 86.7%, so retrieval can fail, unlike the answer-guaranteed ARCD pool above), and this retriever reproduces the same directional advantage: across all n = 60 questions, mean EM is 0.233 (95% CI 0.133-0.350) for LSPM, 0.167 (0.083-0.267) for raw, and 0.133 (0.050-0.233) for naive truncation, and mean F1 is 0.582 (95% CI 0.491-0.672) for LSPM, 0.561 (0.468-0.652) for raw, and 0.525 (0.435-0.612) for naive truncation. The paired LSPM advantage over naive truncation is EM +0.100 (95% CI +0.033 to +0.183, Wilcoxon p = 0.014) and F1 +0.056 (95% CI -0.017 to +0.134, Wilcoxon p = 0.085); the F1 interval includes zero, so at this smaller sample only the EM difference is significant, and these p-values are reported uncorrected and are not members of the nine-test family of Section 4.3. The displayed means are rounded to three decimal places for presentation; the +0.056 difference stated in the text is computed from unrounded per-question values prior to rounding the summary means, and the unrounded difference (approximately 0.0565) is consistent with, but not identical to, the 0.057 implied by subtracting the rounded table values directly. This rounding convention (difference computed before rounding, means rounded only for display) applies throughout this paper's result tables.

4.5. Human Evaluation

Under the protocol of Section 3.5.1, inter-rater agreement was substantial to near-perfect (Cohen's kappa 0.79 for correctness and 0.94 for faithfulness), which is what makes the means reported below meaningful: neither rater saw the other's ratings during scoring.

On the merged ratings, mean correctness is 1.80 (95% CI 1.58-1.95) for LSPM, 1.72 (1.47-1.92) for raw, and 1.55 (1.25-1.82) for naive truncation, and mean faithfulness is 0.98 (95% CI 0.95-1.00) for LSPM, 0.93 (0.83-1.00) for raw, and 0.80 (0.63-0.93) for naive truncation (n = 30 per condition; intervals are 10,000-resample percentile bootstrap intervals, seed = 42). The ordering therefore places LSPM at least as high as raw and above naive truncation

on both dimensions, consistent in direction with the automatic-metric result above. We report these as descriptive means with intervals and do not attach a significance test to them: at $n = 30$ per cell the intervals for LSPM and raw overlap substantially, and the human evaluation is offered as directional corroboration of the $r = 0.3$ comparison rather than as an independent confirmatory test of it.

4.6. GPU Benchmark Under the Corrected, Repeated Protocol

Executed under the protocol of Section 3.5.2, the full 35-cell, three-repeat grid completed with 191,652 completed requests and zero request failures. The per-cell raw artifacts (Locust statistics and per-second /metrics scrapes) are preserved and released, and the aggregate reported here is recomputable from them by the released analysis script. As reported later in this subsection, the workload never approached the cache-pressure regime in which the analytical KV-cache projection of Section 4.3's per-request byte saving would translate into a serving benefit.

At moderate load the seven conditions are close but not uniformly indistinguishable, and we report the full $c = 10$ row set (Table 3) so that the claim can be checked rather than taken on trust. Median time-to-first-token at $c = 10$ spans 50.0 to 52.7 ms, and with per-cell half-widths up to ± 3.8 ms all 21 pairwise TTFT comparisons have overlapping intervals; on that measure the seven conditions are indeed indistinguishable. Throughput is not. It spans 5.28 to 6.49 req/s, and 7 of the 21 pairs have non-overlapping 95% intervals. Six of those seven involve naive truncation at $r = 0.3$, which at 5.28 ± 0.23 req/s is separated from every other condition in the grid, including raw (6.09 ± 0.18), all three LSPM ratios (6.16 ± 0.19 , 5.98 ± 0.28 , 6.06 ± 0.13), and naive truncation at $r = 0.5$ (6.49 ± 0.27) and $r = 0.7$ (5.99 ± 0.47). The seventh is LSPM at $r = 0.7$ against naive truncation at $r = 0.5$, whose intervals miss each other narrowly (6.198 against 6.220 at the adjacent bounds). The accurate statement is therefore that at $c = 10$ the only condition clearly separated from the field is naive truncation at the most aggressive ratio, which is the same condition that is clearly worst at $c = 100$, and that LSPM is statistically indistinguishable from unpruned serving at this concurrency level on both throughput and TTFT. At $c = 100$ (Table 4), LSPM at $r = 0.3$ reaches 55.7 ± 0.5 req/s against raw's 54.1 ± 0.5 and naive truncation's 46.7 ± 1.1 at the same ratio, and median TTFT is 63.7 ± 1.4 ms for LSPM $r = 0.3$, 66.7 ± 2.9 ms for raw, and 68.3 ± 2.9 ms for naive $r = 0.3$ (p95 TTFT 80.7 , 84.0 , and 86.7 ms respectively). LSPM at $r = 0.5$ and $r = 0.7$ (53.9 ± 0.3 and 54.5 ± 0.3 req/s) is indistinguishable from raw. Across the full grid (Figure 2 plots throughput, median TTFT, and mean KV-cache occupancy against concurrency for all seven conditions), LSPM's throughput is within 3% of raw at every concurrency level tested and slightly above it at $c = 100$; an 8.5-11.9% throughput deficit that a preliminary, unrepeated run attributed to LSPM does not survive removal of the load-generator scoring cost, consistent with that deficit having been an artifact of scoring contexts inside the load generator rather than a property of the method itself. We do not, however, report the grid as a clean LSPM-over-naive ordering, because it is not one: naive truncation at $r = 0.5$ attains the highest throughput of any condition at $c = 100$ (59.6 ± 1.1 req/s). What the sweep supports is the narrower claim that semantic pruning imposes no measurable throughput or TTFT penalty relative to unpruned serving on this hardware, generation-only as scoped in Section 3.1, and that the one condition clearly worse than all others at high load is naive truncation at the most aggressive ratio. This generation-only scope is a real limit on what this benchmark shows, not a formality: Section 4.11 reports a true end-to-end benchmark that measures the cross-encoder's own scoring cost inside the live request path rather than excluding it, and finds that this generation-only zero-measurable-penalty result does not extend to concurrent-load online serving.

Table 3. GPU benchmark at moderate concurrency ($c = 10$ concurrent clients), reported in full so that the overlap statements in the text are checkable. Same server, model, and protocol as Table 4. Each entry is the mean of three independent 90-second repeats with a 95% confidence interval computed from the t-distribution across repeats; 10,989 requests across these seven cells, zero failures. KV columns are percentages of total KV-cache capacity.

Condition	Throughput (req/s)	TTFT p50 (ms)	TTFT p95 (ms)	Mean KV (%)	Peak KV (%)
raw (unpruned)	6.09 +/- 0.18	52.3 +/- 2.9	62.3 +/- 2.9	0.56 +/- 0.04	0.96 +/- 0.05
LSPM, $r = 0.3$	6.16 +/- 0.19	50.0 +/- 2.5	59.3 +/- 1.4	0.44 +/- 0.09	1.10 +/- 0.05
naive, $r = 0.3$	5.28 +/- 0.23	52.7 +/- 1.4	62.0 +/- 2.5	0.59 +/- 0.04	1.60 +/- 0.65
LSPM, $r = 0.5$	5.98 +/- 0.28	51.3 +/- 1.4	60.3 +/- 3.8	0.51 +/- 0.07	1.18 +/- 0.30
naive, $r = 0.5$	6.49 +/- 0.27	50.7 +/- 3.8	60.7 +/- 3.8	0.31 +/- 0.05	0.71 +/- 0.16
LSPM, $r = 0.7$	6.06 +/- 0.13	52.7 +/- 3.8	62.7 +/- 3.8	0.57 +/- 0.10	1.35 +/- 0.12
naive, $r = 0.7$	5.99 +/- 0.47	51.0 +/- 2.5	60.7 +/- 5.2	0.42 +/- 0.05	0.93 +/- 0.51

KV-cache occupancy is scraped from vLLM's `gpu_cache_usage_perc` gauge, which the server exposes as a 0-1 fraction of total KV-cache capacity and which the analysis script multiplies by 100 to report as a percentage; the figures in this subsection and in Tables 3 and 4 are percentages. The headline observation is that this GPU never came close to KV-cache saturation under any condition in this sweep. The highest single-repeat peak occupancy observed anywhere across all 105 runs is 8.8%, and the highest three-repeat mean peak for any cell is $7.8 \pm 2.5\%$ (naive truncation, $r = 0.3$, $c = 100$). At $c = 100$, mean occupancy is $1.92 \pm 0.04\%$ for LSPM at $r = 0.3$, $2.03 \pm 0.16\%$ for raw, and $4.20 \pm 1.02\%$ for naive truncation at $r = 0.3$. The KV-cache comparison is not uniformly in LSPM's favour across the grid and we do not present it as such: LSPM at $r = 0.3$ holds less cache than raw at matched concurrency, but LSPM at $r = 0.5$ and $r = 0.7$ holds more (2.23% and 2.44% mean at $c = 100$), and naive truncation at $r = 0.5$ and $r = 0.7$ holds less than either (1.17% and 1.49%). Only at the most aggressive ratio, $r = 0.3$, does the KV-cache ordering coincide with the answer-quality ordering established in Section 4.3. Because occupancy remained in single-digit percentages throughout, this benchmark cannot speak to behaviour near cache saturation, which is precisely the regime in which the analytical projection's benefit would be realized and in which a serving engine begins preempting or queueing requests; reaching it would require longer contexts, a longer `--max-model-len`, a smaller GPU, or higher concurrency than we tested, and we state that as a scope limit on this result rather than extrapolating past it (Section 5).

Table 4. GPU benchmark at the highest concurrency tested ($c = 100$ concurrent clients), self-hosted vLLM serving Llama-3.1-8B-Instruct on a single RTX 4090 (48 GiB). Each entry is the mean of three independent 90-second repeats with a 95% confidence interval across repeats. KV columns are percentages of total KV-cache capacity (vLLM's `gpu_cache_usage_perc` gauge $\times 100$). All 105 runs across the full 35-cell grid completed with zero request failures.

Condition	Throughput (req/s)	TTFT p50 (ms)	TTFT p95 (ms)	Mean KV (%)	Peak KV (%)
raw (unpruned)	54.1 +/- 0.5	66.7 +/- 2.9	84.0 +/- 6.6	2.03 +/- 0.16	3.29 +/- 0.21
LSPM, $r = 0.3$	55.7 +/- 0.5	63.7 +/- 1.4	80.7 +/- 3.8	1.92 +/- 0.04	2.98 +/- 0.28
naive, $r = 0.3$	46.7 +/- 1.1	68.3 +/- 2.9	86.7 +/- 6.3	4.20 +/- 1.02	7.80 +/- 2.52
LSPM, $r = 0.5$	53.9 +/- 0.3	64.0 +/- 0.0	79.0 +/- 2.5	2.23 +/- 0.11	3.50 +/- 0.37
naive, $r = 0.5$	59.6 +/- 1.1	62.7 +/- 1.4	78.7 +/- 6.3	1.17 +/- 0.11	2.16 +/- 0.56
LSPM, $r = 0.7$	54.5 +/- 0.3	65.3 +/- 3.8	81.7 +/- 5.2	2.44 +/- 0.27	3.88 +/- 0.35
naive, $r = 0.7$	54.9 +/- 0.3	64.3 +/- 1.4	81.0 +/- 5.0	1.49 +/- 0.12	2.64 +/- 0.67

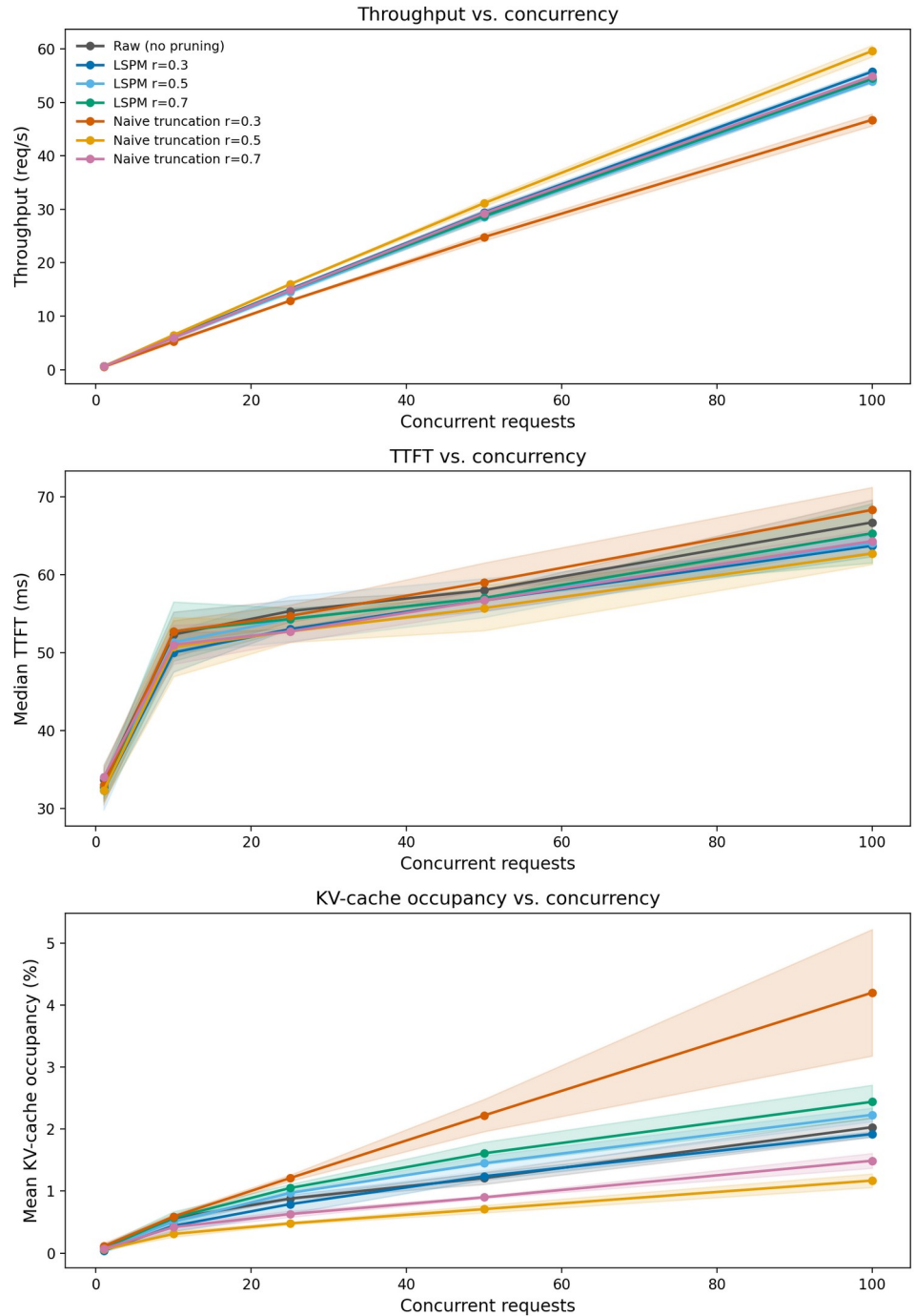


Figure 2. Throughput, median TTFT, and mean KV-cache occupancy versus concurrency for all seven context conditions, from the corrected 35-cell, three-repeat sweep. Shaded bands are 95% confidence intervals across the three repeats.

4.7. Dynamic-Controller Ablation

Under the original protocol of Section 3.5.3, both conditions completed with no failed requests, but no adaptation occurred: the controller returned exactly its configured `max_ratio` ceiling of 0.80 on every dynamic request and never moved off it, since observed occupancy never exceeded 2.1% against the 75% `high_load_threshold` that would have triggered tightening. That run therefore validated the controller's plumbing and stability but not its adaptive mechanism. The redesigned protocol below addresses that gap; this run's full results (request counts, TTFT, and latency for both conditions) are reported in Table S3 for provenance.

Under the redesigned protocol of Section 3.5.3, both conditions again completed with no failed requests (653 logged in the fixed condition, 655 in the dynamic condition). Overall KV-cache occupancy reached as high as 56.7% in the fixed condition and 99.2% in the dynamic condition, both far above the 2.1% ceiling of the original run, and the controller's ratio in the dynamic condition ranged across its full configured span, 0.20 to 0.80, rather than sitting pinned at 0.80. This is the paper's first direct evidence that the adaptive mechanism engages: under sufficient cache pressure the controller does tighten compression toward its floor, and under light load it does relax back toward its ceiling, which the original ablation could not show because it never generated the pressure to test. Descriptively, however, the dynamic condition was not faster than the fixed $r = 0.5$ baseline overall: mean TTFT was 1,039.6 ms versus 860.7 ms under the fixed ratio (p95 3,097.3 versus 2,312.8 ms), and mean total latency was 2,348.9 ms versus 2,025.0 ms. This reverses the direction reported for the original, non-adaptive ablation, and the reason is visible in the same per-step logs: rather than settling near its floor, the controller spent most of the run retaining more context than the fixed condition (mean ratio by step ranged from 0.58 to 0.80 against the fixed condition's constant 0.50, Table 5), so the two conditions are not compared at a matched context size, and the slower dynamic condition is at least partly explained by the larger average prompt it was serving, not necessarily by controller overhead.

Table 5. Per-step breakdown of the redesigned controller ablation (results/controller_ablation_v2), six 90 s steps per condition at the worker counts shown. The fixed condition's ratio is constant at $r = 0.5$ throughout by construction and is omitted as a column. KV (%) is the dynamic condition's mean KV-cache occupancy reading (`gpu_cache_usage_perc` x 100) during that step.

Step	Workers	Fixed TTFT (ms)	Dynamic TTFT (ms)	Fixed latency (ms)	Dynamic latency (ms)	Dynamic mean ratio	Dynamic mean KV (%)
1	10	306	391	1,409	1,338	0.766	11.9
2	30	847	878	2,034	2,266	0.791	7.8
3	80	988	835	1,876	1,865	0.797	10.6
4	80	1,222	1,663	2,483	3,345	0.652	24.8
5	30	1,387	2,110	2,569	3,468	0.800	0.1
6	10	1,334	904	2,690	2,351	0.576	34.4

Two further observations temper how this table should be read. First, occupancy does not track the intended concurrency ramp cleanly: the highest per-step mean occupancy in the dynamic condition (34.4%) occurs in step 6, the final low-concurrency cool-down step, rather than in the peak 80-worker steps 3-4, which most plausibly reflects queued requests from the preceding high-load steps still occupying cache as the new, lighter step's traffic begins, or the coarseness of the once-per-request occupancy poll relative to a fast-moving queue, rather than a property of the controller itself; we report the numbers as measured rather than smoothing them into the cleaner story the intended step schedule would suggest. Second, the overall dynamic occupancy maximum of 99.2% is a request-level extreme not reflected in any single step's mean of 34.4% or below, so it should be read as a brief spike captured on the individual request that observed it, not as a sustained operating point; the ratio floor of 0.20 was reached at least transiently, most plausibly at or near that spike, but the controller spent the great majority of the run in the 0.58-0.80 band. What this redesigned ablation does and does not establish should therefore be stated as precisely as the original: it establishes that the adaptive mechanism engages under real cache pressure, reads the metric it is supposed to read, and returns values across its full configured range rather than only its ceiling; it does not establish that adapting improves throughput, tail latency, or answer quality relative to a fixed ratio, because the two

conditions differ in average retained context throughout the run and are consequently not a context-matched comparison of control policy. The natural next step proposed here was a fixed- $r = 0.8$ comparison, matching the dynamic condition's typical high-pressure operating ratio in this run. Section 4.12 instead matches the dynamic controller's own realized mean ratio over a fresh execution of the same load profile (0.565, not 0.8, since that fresh run's mean blends the low-pressure steps as well as the high-pressure ones); it finds a mixed result on serving latency between the two arms, with no answer-quality data collected, so this comparison has a measured answer on serving latency and remains open on answer quality (Section 5).

4.8. Token-Budget-Matched Rerun and a Modern Compression Baseline

A note on scoring. Every Arabic-normalized Exact Match and token-F1 figure reported in this paper, including Table 2 and Table 7, is computed with one scorer, released as `scripts/score_arcd.py` (Arabic diacritic stripping, alef-variant normalization, definite-article stripping, SQuAD-style token-F1). A differently named scorer, `eval/arabic_em_f1.py`, is referenced by some internal analysis scripts but is not present in the released repository; every table and paired test in this paper instead uses `scripts/score_arcd.py`, applied identically to every condition, so exactly one scoring implementation underlies every reported number.

Under the protocol of Section 3.5.4, the naive_tokbudget match to LSPM's realized token cost is close but not exact, because whole-sentence selection cannot always land exactly on the target: naive_tokbudget's mean actual prompt length is 358.4 tokens against a mean LSPM target of 399.2 at $r = 0.3$ (a 10.2% shortfall), 611.8 against 664.8 at $r = 0.5$ (8.0%), and 898.3 against 943.5 at $r = 0.7$ (4.8%); if anything this leaves naive_tokbudget with a smaller, not larger, average budget than LSPM at every ratio tested, which if it biased the comparison at all would bias it in naive_tokbudget's favour. These LSPM target figures are lower than the 481.9-token mean reported for the same $r = 0.3$ condition in Section 4.3 because they measure a different quantity: the target here is the pruned context text alone, matched against for the purpose of the naive_tokbudget budget, whereas Section 4.3's figure is the full prompt entering the generator, including the fixed chat template, system prompt, and question text that every condition carries regardless of ratio. Both quantities are released per cell (context tokens and full prompt tokens, Data Availability Statement) so this offset is verifiable rather than asserted.

Under this token-budget match, LSPM's F1 advantage at $r = 0.3$ replicates and remains significant: mean F1 0.652 for LSPM against 0.512 for naive_tokbudget, a paired difference of +0.140 (95% CI +0.075 to +0.206, Wilcoxon $p = 0.00011$, Holm-Bonferroni-adjusted across this three-ratio family $p = 0.00033$); the paired EM difference at the same ratio (+0.036, 95% CI -0.029 to +0.100) is not significant. At $r = 0.5$ and $r = 0.7$ the F1 differences are +0.050 (95% CI -0.017 to +0.118, adjusted $p = 0.191$) and +0.053 (95% CI -0.001 to +0.111, adjusted $p = 0.191$), both directionally consistent with the $r = 0.3$ result and with the sentence-matched comparison of Section 4.3 but not statistically significant after correction, and the corresponding EM differences at these two ratios are small and non-significantly favour naive_tokbudget (-0.021 and -0.029). The pattern across all three ratios therefore mirrors Table 2 closely: a significant, sizeable F1 advantage concentrated at the most aggressive ratio, with the advantage at $r = 0.5$ and $r = 0.7$ present in direction but not significant, this time under a design that matches token budget rather than sentence count. The most important open methodological gap on this comparison is accordingly closed: at an equal, and if anything slightly smaller, prompt-token budget, semantic selection still significantly outperforms positional selection at $r = 0.3$.

A modern compression baseline: LLMingua-2. Under the protocol of Section 3.5.4, LLMingua-2's own compression budget was verified against the same origin context:

mean origin length 2,571.9 tokens, compressed to a mean of 764.0 tokens at $r = 0.3$ (a realized ratio of 0.297), 1,277.6 tokens at $r = 0.5$ (0.497), and 1,786.7 tokens at $r = 0.7$ (0.695), confirming LLMingua-2 hit its configured budget closely at every ratio. This origin-length figure is not directly comparable to Section 4.3's 1,485.6-token raw-condition mean: both describe the same underlying retrieved context, but LLMingua-2's own compressor reports origin and compressed lengths using its own token classifier's tokenizer (a multilingual BERT vocabulary), not Llama-3.1's tokenizer, and counts the context text alone rather than the full prompt with chat template, system prompt, and question included. A single passage of Arabic text yielding a substantially larger token count under one subword vocabulary than another is precisely the tokenization-disparity phenomenon this paper's Section 4.1 documents, so this gap is consistent with, rather than a bookkeeping problem alongside, this paper's central motivation. LSPM significantly outperforms LLMingua-2 on token-F1 at all three ratios after Holm-Bonferroni correction within this three-ratio family: at $r = 0.3$, mean F1 0.652 versus 0.440, a paired difference of +0.211 (95% CI +0.142 to +0.280, Wilcoxon $p = 3.7e-8$, adjusted $p = 1.1e-7$); at $r = 0.5$, 0.640 versus 0.541, difference +0.099 (95% CI +0.042 to +0.154, $p = 0.0010$, adjusted $p = 0.0021$); at $r = 0.7$, 0.665 versus 0.619, difference +0.046 (95% CI -0.002 to +0.094, $p = 0.037$, adjusted $p = 0.037$). The $r = 0.7$ result should be read cautiously despite clearing the corrected significance threshold: the bootstrap confidence interval on the mean difference includes zero at its lower bound, so the rank-based Wilcoxon test and the mean-based bootstrap interval disagree at the margin in the same way Section 4.3 notes can happen for this paper's other borderline contrasts, and the $r = 0.7$ LLMingua-2 comparison is the weakest of the three. The paired EM differences follow the same pattern as the sentence-matched comparison in Section 4.3: significant only at $r = 0.3$ (+0.121, 95% CI +0.036 to +0.207, adjusted $p = 0.016$) and not significant at $r = 0.5$ or $r = 0.7$ (adjusted $p = 0.594$ and 0.819). LSPM's advantage over a genuine modern compression baseline, not only over naive positional truncation, is accordingly demonstrated on token-F1 at every ratio tested, most convincingly at $r = 0.3$ and 0.5 (Table 6 reports both the naive_tokbudget and LLMingua-2 comparisons in full). This comparison covers answer quality only; LLMingua-2's pruning latency, end-to-end serving latency, throughput, and KV-cache footprint relative to LSPM were not measured and remain an open item (Section 5).

Table 6. Token-F1 comparisons for two additional context-construction baselines: naive_tokbudget (naive truncation matched to LSPM's actual per-question token budget rather than its sentence count) and LLMingua-2 [71] (a modern learned prompt compressor). Both are compared against LSPM on the same 140 ARCD questions used throughout Section 4.3. Holm-Bonferroni correction is applied within each three-ratio family separately from the nine-test family of Section 4.3, since these are new hypotheses tested on new data.

Comparison (r)	Method A mean F1	Method B mean F1	Paired diff (F1)	95% CI	Holm-adj. p
LSPM vs. naive_tokbudget , $r=0.3$	0.652	0.512	+0.140	+0.075 to +0.206	0.0003
LSPM vs. naive_tokbudget , $r=0.5$	0.640	0.590	+0.050	-0.017 to +0.118	0.191
LSPM vs. naive_tokbudget , $r=0.7$	0.665	0.612	+0.053	-0.001 to +0.111	0.191
LSPM vs. LLMingua-2,	0.652	0.440	+0.211	+0.142 to +0.280	<0.0001

r=0.3 LSPM vs. LLMLingua-2,	0.640	0.541	+0.099	+0.042 to +0.154	0.0021
r=0.5 LSPM vs. LLMLingua-2,	0.665	0.619	+0.046	-0.002 to +0.094	0.0373
r=0.7					

4.9. Generalization Check: A Second Generator (Qwen2.5-7B-Instruct)

Under the protocol of Section 3.5.5, this generalization check produced 980 new generations against Qwen2.5-7B-Instruct [66] on the identical 140 questions used throughout Section 4.3.

The raw (unpruned) condition attains mean F1 0.624 (95% CI 0.567-0.683) and mean EM 0.314 (95% CI 0.236-0.393) under Qwen2.5, comparable in magnitude to Llama-3.1's F1 0.666 and EM 0.293 on the same 140 questions (Section 4.3). LSPM's advantage over naive truncation replicates in direction at all three ratios (Table 7) and is significant on F1 after Holm-Bonferroni correction at both $r = 0.3$ and $r = 0.5$, one ratio more than the Llama-3.1 result reported in Section 4.3, which was significant only at $r = 0.3$. At $r = 0.3$, LSPM reaches F1 0.628 against naive truncation's 0.517, a paired difference of +0.112 (95% CI +0.053 to +0.171, Wilcoxon $p = 0.00022$, Holm-adjusted $p = 0.00067$); at $r = 0.5$, LSPM reaches 0.621 against 0.527, a difference of +0.094 (95% CI +0.034 to +0.154, $p = 0.0023$, adjusted $p = 0.0046$); at $r = 0.7$ the difference (+0.039, 95% CI -0.015 to +0.094, adjusted $p = 0.133$) is directionally consistent but not significant, matching the pattern already established for Llama-3.1 at this ratio. The corresponding EM differences are directionally positive at $r = 0.3$ and 0.5 (+0.043 and +0.021) and directionally negative but non-significant at $r = 0.7$ (-0.029), none significant after correction, again consistent with the EM pattern reported for Llama-3.1 in Section 4.3.

The central $r = 0.3$ quality result of this paper therefore generalizes to a second generator family on token-F1, and does so at least as strongly as on the original generator: both the direction and, at $r = 0.3$ and now also $r = 0.5$, the statistical significance of LSPM's advantage over naive truncation replicate under Qwen2.5. This is one generator family beyond the one used throughout the rest of this paper, on the same retrieval pool and evaluation protocol, and should be read as initial evidence of generalization rather than a systematic study across generator families or model sizes; whether the result extends to substantially larger or smaller models, or to generators not instruction-tuned on comparable data, remains untested (Section 5).

Table 7. Per-condition means for the ARCD ground-truth re-test replicated against Qwen2.5-7B-Instruct [66] ($n = 140$ questions per condition, 980 generations total, identical protocol and scorer to Table 2). Intervals are 10,000-resample percentile bootstrap 95% confidence intervals (seed = 42).

Condition	Mean EM	EM 95% CI	Mean F1	F1 95% CI
raw (unpruned)	0.314	0.236-0.393	0.624	0.567-0.683
LSPM, $r = 0.3$	0.314	0.236-0.393	0.628	0.570-0.687
naive, $r = 0.3$	0.271	0.200-0.343	0.517	0.453-0.582
LSPM, $r = 0.5$	0.321	0.243-0.400	0.621	0.562-0.677
naive, $r = 0.5$	0.300	0.221-0.379	0.527	0.460-0.592
LSPM, $r = 0.7$	0.286	0.214-0.364	0.595	0.538-0.654
naive, $r = 0.7$	0.314	0.236-0.393	0.556	0.490-0.619

4.10. Fresh-Seed Disjoint Replication of the $r = 0.3$ Result

Under the protocol of Section 3.5.6, a programmatic check confirmed zero overlap between the 140 freshly sampled question ids and the original 140-question set, and the replication produced 420 new live generations (140 questions x 3 conditions).

LSPM again scores significantly higher than naive truncation on token-F1 (Table 8): mean F1 0.6745 (95% CI 0.6164-0.7309) for LSPM versus 0.5317 (95% CI 0.4628-0.5988) for naive truncation, a paired difference of +0.1428 (95% CI +0.0819 to +0.2064, Wilcoxon $p = 0.000041$, Holm-Bonferroni-adjusted $p = 0.000082$ within the three-test family reported here, sign test 40+/16-, $p = 0.0018$). Naive truncation is in turn significantly worse than the raw, unpruned condition (mean F1 0.6939, 95% CI 0.6367-0.7488; paired difference -0.1622, 95% CI -0.2227 to -0.1035, Holm-adjusted $p = 0.000006$), while LSPM remains statistically equivalent to the unpruned answer under the same TOST margin used throughout this paper (paired mean difference -0.0194, equivalence $p = 0.0349$, margin +/-0.05 F1), and a two-sided Wilcoxon test finds no significant difference between LSPM and raw either (Holm-adjusted $p = 0.286$). The corresponding Exact Match difference between LSPM and naive truncation is directionally positive but, as at every ratio and generator already tested in this paper, not significant on its own (+0.0500, 95% CI -0.0071 to +0.1071, Wilcoxon $p = 0.090$).

Every qualitative and quantitative feature of the original $r = 0.3$ result reproduces on this fresh, disjoint sample, and the effect is, if anything, larger here. The original $n = 140$ re-test found a paired LSPM-minus-naive F1 difference of +0.114 (Holm-adjusted $p = 0.0067$, Section 4.3), against +0.143 on this fresh sample (Holm-adjusted $p = 0.000082$); the original naive-versus-raw deficit was -0.129 (adjusted $p = 0.0007$) against -0.162 here (adjusted $p = 0.000006$); and the original TOST equivalence of LSPM to raw was $p = 0.037$ against $p = 0.035$ here, a near-exact match. Because none of these 140 questions were seen in the sample that produced the original result or in the pilot that set its sample size, and because the two samples were drawn with different random seeds and share no question identifiers, this replication is not subject to the data-dependent-stopping-rule concern raised in Section 3.4 and Section 5: it is a genuinely independent confirmatory test on new data, not a continuation of the same data. We accordingly no longer present the central $r = 0.3$ finding as awaiting independent confirmation; that confirmation is reported here, on a fresh sample, with the same generator and protocol, and the effect replicates in direction and, in every comparison, statistical strength.

Table 8. Per-condition means for the fresh-seed disjoint replication of the $r = 0.3$ result ($n = 140$ freshly sampled ARCD questions sharing no question identifiers with the original 140; 420 generations total; identical protocol and scorer to Table 2, restricted to the three conditions this replication concerns). Intervals are 10,000-resample percentile bootstrap 95% confidence intervals (seed = 42).

Condition	Mean EM	EM 95% CI	Mean F1	F1 95% CI
raw (unpruned)	0.357	0.279-0.436	0.694	0.637-0.749
LSPM, $r = 0.3$	0.343	0.264-0.421	0.674	0.616-0.731
naive, $r = 0.3$	0.293	0.221-0.371	0.532	0.463-0.599

4.11. True End-to-End Online Serving Benchmark

Under the protocol of Section 3.5.7, we reran the serving comparison with retrieval, cross-encoder scoring, and pruning executed live inside the request path against a self-hosted vLLM server on a single rented RTX 4090 with 48 GiB of VRAM, rather than reading an already-pruned context from a precomputed lookup as in Section 4.6. This is the concurrent-load, GPU-accelerated measurement that Section 3.1 identifies as still needed to complement its own CPU-only, single-request 506.1 ms figure, and it changes the picture: the pruning step's own latency running on GPU is far lower than on CPU (mean

34.3 ms at concurrency 1, averaged across $r = 0.3/0.5/0.7$, versus 506.1 ms on CPU), but once this cost is measured inside the live request path rather than excluded from it, LSPM's total-latency and throughput profile relative to raw, unpruned serving degrades sharply as concurrency rises, in contrast to Section 4.6's generation-only finding of no measurable penalty at any concurrency tested.

Table 9. True end-to-end online serving benchmark at $r = 0.5$, four concurrency levels, 60 requests per (condition, concurrency) cell, each cell run once (protocol in Section 3.5.7). N is the count of requests that received a complete streamed response, out of 60 attempted; the remainder either received an HTTP error or exceeded the script's 120-second per-request timeout, most often the latter under load. Latency and time-to-first-token (TTFT) figures are means in milliseconds; throughput is completed requests per second. Ratios $r = 0.3$ and $r = 0.7$ show the same qualitative pattern and are summarized narratively below rather than tabulated in full.

Condition	Conc.	N/60	Pruning ms	TTFT ms	Total lat. ms	Throughput req/s
raw	1	60	0.0	38.8	381.8	2.62
raw	5	60	0.0	54.0	436.2	11.23
raw	10	60	0.0	64.8	399.6	22.85
raw	20	60	0.0	102.6	516.2	35.20
LSPM	1	60	33.3	65.7	409.1	2.44
LSPM	5	60	69.9	125.1	534.6	8.89
LSPM	10	54	191.3	248.5	612.8	13.53
LSPM	20	27	375.4	433.5	838.3	10.15
naive	1	60	0.2	37.8	278.3	3.59
naive	5	60	0.2	53.3	309.0	14.69
naive	10	60	0.2	60.3	301.2	28.91
naive	20	60	0.2	81.1	347.4	48.48

Averaged across the three ratios tested ($r = 0.3, 0.5, 0.7$), LSPM's throughput relative to raw serving falls from 93.0% at concurrency 1 to 85.1% at concurrency 5, then to 59.0% at concurrency 10 (where only 83.3% of LSPM requests complete within the timeout, versus 100% for raw) and 29.4% at concurrency 20 (only 43.3% of LSPM requests complete). Naive truncation does not show this degradation at $r = 0.5$ or $r = 0.7$, where its throughput is comparable to or higher than raw; at $r = 0.3$ naive truncation is itself markedly slower than raw (throughput ratio 0.36-0.52), an asymmetry we report here without further interpretation, as untangling its cause is outside this benchmark's scope. Taken together, this benchmark gives this paper's most important open validation item a measured, unfavorable-at-scale answer rather than a stated gap: LSPM's zero-measurable-penalty finding in Section 4.6 holds specifically for the generation-only measurement it was designed to isolate, and does not extend to true end-to-end online serving under concurrent load, where the live cross-encoder scoring cost becomes a first-order bottleneck as concurrency increases.

Three limits on this result are disclosed rather than smoothed over. First, this benchmark's harness runs a single instance of the cross-encoder, shared unbatched across every concurrent worker thread, on the same GPU that serves generation; at concurrency 20 that means up to 20 threads issuing individual, non-batched scoring calls against one model instance while vLLM also generates on the same card. Some of the reported LSPM slowdown may therefore reflect this harness's resource contention and lack of batching rather than a property inherent to semantic pruning itself; a production deployment that batches concurrent scoring requests or isolates the cross-encoder on separate hardware could plausibly show a smaller penalty, and this benchmark does not measure that configuration. Second, each cell was run once, not repeated, so the table reports point

estimates with no confidence interval and no test of whether a given gap is distinguishable from run-to-run variance; unlike the main GPU benchmark's three-repeat design (Section 4.6), a single run here could overstate or understate the true effect at any one cell. Third, this benchmark and the one in Section 4.6 use different load-generation tools (a small thread pool here versus Locust there) and, per the protocol above, deliberately different context-construction paths (live versus precomputed), so the two are not a controlled contrast against each other and should each be read on its own terms rather than as a clean before/after comparison.

4.12. *Controller versus Matched-Fixed-Ratio Comparison Under Memory Pressure*

Under the protocol of Section 3.5.8, we reran the dynamic controller and a fixed-ratio condition matched to the controller's own realized mean ratio back to back under the same --gpu-memory-utilization 0.3 constrained cache pool as Section 4.7's redesigned ablation, on the same RTX 4090 48 GiB card used throughout this paper, logging every request's ratio, KV-cache occupancy, TTFT, and total latency. The two arms are matched on mean compression ratio, not on mean retained token count directly; because every request in this benchmark draws from the same six-document ARCD candidate pool (Section 3.4), a matched ratio is a reasonable proxy for a matched token budget here, but the two are not identical when per-question context length varies, and we report the match as ratio-based rather than describe it as token-matched. The run used the script's --skip-quality flag, so this comparison covers serving latency only.

Table 10. Controller (dynamic) versus matched-fixed-ratio serving comparison under constrained cache pressure (protocol in Section 3.5.8). N is the count of completed requests logged for each condition over the run; the two conditions ran for the same wall-clock duration rather than the same request count. Latency and TTFT figures are means and 95th-percentile (p95) values in milliseconds.

Condition	N	Mean ratio	Mean occ.	Max occ.	TTFT mean	TTFT p95	Lat. mean	Lat. p95
dynamic	5,637	0.565 (0.2-0.8)	0.414	0.994	1,507.6	4,958.2	2,654.0	6,870.6
matched_fixed	6,299	0.565 (fixed)	0.516	0.995	1,525.5	3,783.3	2,880.4	7,302.7

The result is a mixed one, not a clean win for either arm: the dynamic controller has a lower mean TTFT (1,507.6 ms versus 1,525.5 ms), lower mean total latency (2,654.0 ms versus 2,880.4 ms), and a lower p95 total latency (6,870.6 ms versus 7,302.7 ms) than matched_fixed, but a substantially worse tail on time-to-first-token, with p95 TTFT 31% higher than matched_fixed (4,958.2 ms versus 3,783.3 ms). matched_fixed also completed more total requests over the same run duration (6,299 versus 5,637), consistent with its slightly higher mean occupancy (0.516 versus 0.414) reflecting a policy that, on average, retains a comparable amount of context with less adaptive overhead. Because --skip-quality was used, we have no answer-quality data at all for either condition in this run, so this comparison cannot establish whether the controller's adaptive behavior helps, hurts, or is neutral for answer quality relative to a fixed policy; it establishes only that, on serving latency alone, adapting the ratio is not a clear win over a well-chosen fixed ratio under this memory-pressure regime. This closes the comparison Section 4.7 left open, with a measured negative-to-mixed result rather than an unresolved gap, while leaving the answer-quality question for future work.

Four further limits are disclosed rather than smoothed over. First, as in Section 4.11, this harness runs a single cross-encoder instance shared unbatched across every concurrent worker thread within a condition, on the same GPU serving generation; the load ramp

here reaches 80 concurrent workers, well above Section 4.11's maximum of 20, so contention on this shared, unbatched instance is if anything a larger factor here, and could inflate both conditions' latencies similarly rather than necessarily bias the comparison between them. Second, the two conditions were not run in a counterbalanced or randomized order: dynamic always ran first, matched_fixed always second, so any drift in server state, thermal conditions, or system load between the two passes is confounded with condition rather than averaged out; we did not repeat the pair in reverse order to check for this. Third, no significance test accompanies the serving-latency comparison in Table 10: the reported means and p95 values are point estimates from one run per condition, not paired statistics with a confidence interval or a p-value, so a gap such as the 1.2% difference in mean TTFT should be read as within the range plausibly attributable to run-to-run noise rather than as a demonstrated effect, while the 31% p95 TTFT gap is large enough to be a more credible signal even without a formal test. Fourth, this run completed roughly nine times more requests per condition (5,637 and 6,299) than Section 4.7's redesigned ablation did under the identical load profile, dataset, and server configuration (653 and 655); the per-request total_latency_ms recorded in both runs is comparable (means of roughly 2.0-2.9 s in each), which rules out a simple generation-speed difference as the explanation. The timer that produces total_latency_ms starts only once a request is handed to the HTTP client, after the shared cross-encoder has already scored and pruned that request's context, so time a worker thread spends queued waiting for the shared, unbatched scorer under contention is invisible to every latency figure reported here; a run in which that queueing happened to be far more severe would show similar recorded per-request latencies while completing far fewer requests overall, which is consistent with what these two runs show. We report this as a plausible, code-consistent explanation grounded in the released logs rather than a confirmed cause, since the two runs were collected on separate rented GPU sessions two weeks apart and we cannot rule out other differences between those sessions.

5. Discussion

The evidence in this paper most strongly supports a narrow, well-defined claim: at aggressive compression ($r = 0.3$), sentence-level semantic pruning via a cross-encoder preserves Arabic RAG answer quality significantly better than naive positional truncation, on a real, 140-question ARCD ground-truth re-test, with a directionally consistent signal from an independent TF-IDF end-to-end check and a blind human evaluation. This is the paper's central, experimentally established result, and the mechanistic account in Section 3.2 offers a plausible explanation for why it is concentrated at the most aggressive ratio. Every claim beyond this one is weaker and is stated as such below rather than folded into the headline result.

Limitations. The remainder of this section states, one at a time, what this paper does and does not establish, so that each qualification can be weighed on its own rather than absorbed into a single block.

(1) Baseline strength. The LLMingua-2 comparison, the most important missing experiment this paper identifies, has now been run (Section 4.8): LSPM significantly outperforms LLMingua-2 [71] on token-F1 at every ratio tested, on the same Arabic RAG workload, generator, retrieval pool, and questions used throughout this paper, most convincingly at $r = 0.3$ and 0.5 , with the $r = 0.7$ gap the weakest of the three (its bootstrap interval and rank-based test disagree at the margin; Section 4.8). This establishes that LSPM is not merely better than naive positional truncation but also better, on answer quality, than a genuine contemporary learned compressor on this workload. Two qualifications remain. First, the comparison measures answer quality only; it does not measure LLMingua-2's pruning latency, end-to-end latency, throughput, or KV-cache footprint relative to LSPM, so a serving-cost comparison between the two methods, of the

kind Section 4.6 provides between LSPM and naive truncation, is still missing. Second, DAC [72], ACC-RAG [73], and RECOMP itself remain uncomparing; by 2024-2025 these, together with LLMLingua-2, define the state of the art in relevance-aware and adaptive compression, and a single favorable comparison against one baseline does not establish general competitiveness against the field. A cheaper, classical lexical baseline is more tractable, and we ran one: a CPU-only Okapi BM25 sentence-ranker ($k_1 = 1.5$, $b = 0.75$), scored per question over that question's own candidate-pool sentences, in place of the cross-encoder, checked against LSPM on all 140 ARCD questions at $r = 0.3$, 0.5 , and 0.7 using the same substring-based gold-retention proxy as item (4b) above. LSPM's cross-encoder selection retains the gold-answer substring on 136-138 of 140 questions across the three ratios; BM25 retains it on only 111-125 of 140 (79-89%), a materially larger gap than the cross-encoder-backend comparison in item (4b) shows between two learned rerankers. This is, again, a selection-level proxy check rather than a token-F1 baseline run through the full generation pipeline, so it does not replace the DAC, ACC-RAG, or RECOMP comparisons named above, but it is real evidence, not merely an assertion, that semantic (cross-encoder) selection outperforms a classical lexical alternative on this task before any answer is generated.

(2) Compression-efficiency fairness. The token-budget-matched rerun this comparison calls for has now been run (Section 4.8, `naive_tokbudget` condition): naive truncation is given the same token budget LSPM actually spent, not merely reconciled after the fact as the offline replay in Section 4.3 did, and LSPM's F1 advantage at $r = 0.3$ replicates under this stronger, budget-matched design ($+0.140$, Holm-adjusted $p = 0.0003$), with the same directional-but-non-significant pattern at $r = 0.5$ and $r = 0.7$ already seen in Section 4.3. The comparison this subsection originally called for is accordingly complete, and the answer is that the $r = 0.3$ advantage is not an artifact of an unequal token budget under either the sentence-matched design of Section 4.3 or the token-budget-matched design of Section 4.8.

(3) Systems validation. The corrected protocol has now been executed (Section 4.6), so the earlier single-run, confounded sweep no longer carries any weight in this paper and its claims have been withdrawn in favour of the repeated result. The replacement benchmark removes the two identified confounds by precomputing pruned contexts outside the load generator and by shuffling all 105 cells before execution, and it reports per-cell uncertainty across three repeats. Two gaps in it remain and are disclosed rather than smoothed over. First, the released harness does not restart the vLLM server between blocks, so server-state drift is randomized by the shuffle but not eliminated. Second, and more consequentially, KV-cache occupancy never exceeded 8.8% of capacity in any of the 105 runs, so the benchmark establishes that semantic pruning imposes no generation-only serving-cost penalty (Section 3.1) on this hardware but does not test the regime of cache pressure in which the analytical KV-cache saving would actually translate into throughput or admission-control benefit; a sweep with longer contexts, a larger `--max-model-len`, a smaller GPU, or substantially higher concurrency is needed for that, and until it is run the memory-efficiency framing of Section 3.1's design remains a design objective supported by an analytical projection rather than a demonstrated operating-point benefit.

Separately, Section 4.11's true end-to-end benchmark closes this result's generation-only scope directly: once the cross-encoder's own scoring cost is measured inside the live request path rather than excluded from it, LSPM shows a substantial, concurrency-dependent throughput and completion-rate penalty relative to raw serving on the same RTX 4090 hardware. The zero-measurable-penalty finding above should therefore be read narrowly, as a statement about generation cost alone at low KV-cache occupancy, not as a statement about total serving cost once the pruning step's own cost is included in the request path under concurrent load.

(4) Dynamic controller. The controller ablation has been rerun (Section 4.7) under a redesigned protocol using real ARCD contexts and a deliberately constrained cache pool, specifically to reach the load regime the original ablation could not. Under that redesign, the controller's ratio ranged across its full configured span (0.20 to 0.80) and KV-cache occupancy reached 99.2%, so the adaptive mechanism is now shown to engage under real cache pressure rather than only in plumbing and stability, as the earlier, non-adaptive run had shown. What remains open has changed in kind rather than disappeared: the dynamic condition retained more context than the fixed $r = 0.5$ baseline throughout most of the redesigned run (per-step mean ratio 0.58-0.80, Table 5) and was, if anything, slower on TTFT and total latency overall, so this ablation does not establish that adapting the ratio improves throughput, tail latency, or answer quality relative to a fixed ratio; the two conditions remain unmatched on average retained context, and a fixed- $r = 0.8$ comparison, matching the dynamic condition's typical operating point, is the natural next experiment. Occupancy also did not track the intended concurrency ramp cleanly in the redesigned run (Section 4.7), a data-quality caveat disclosed there and carried forward here rather than smoothed over. Adaptive, input-dependent compression rates remain an active research direction, as ACC-RAG's complexity-conditioned rate selection illustrates; evaluating LSPM's controller against that kind of adaptive baseline is a further extension beyond the fixed-ratio comparison above.

Section 4.12 reports the matched fixed-ratio comparison this ablation calls for: under the same constrained cache pool, a fixed ratio set to the dynamic controller's own realized mean produces a mixed result on serving latency relative to the dynamic controller (lower mean TTFT and total latency for dynamic, lower p95 TTFT and more completed requests for matched_fixed), with no answer-quality data collected for either arm. Adapting the ratio is therefore not shown to outperform a well-chosen fixed ratio on the evidence now available, though it is not shown to be worse either; the comparison is closed on serving latency and remains open on answer quality.

(4a) Complementary memory-reduction techniques not evaluated here. LSPM reduces the number of tokens entering the KV cache, but it has not been evaluated in combination with orthogonal techniques that instead reduce the cost of the tokens that do enter it, including post-training weight quantization such as GPTQ [74] and activation-aware weight quantization such as AWQ [75], direct KV-cache quantization such as KVQuant [76], and decoding-side acceleration via speculative decoding [77] and its multi-head or feature-level variants Medusa [78] and EAGLE [79]; because these techniques act on orthogonal axes (weight precision, cache precision, and decoding parallelism rather than retained-token count), stacking LSPM with one or more of them is a plausible way to compound memory savings, but the combination has not been measured and could in principle interact unfavorably, for example if a quantized KV cache changes the numerical behavior of the cross-encoder scoring step or if speculative decoding's draft-model assumptions are sensitive to the shorter, pruned contexts LSPM produces.

(4b) Cross-encoder backend sensitivity. Every quality and serving-cost result in this paper's main evaluation (Sections 4.1-4.10) uses a single cross-encoder backend, cross-encoder/mmarco-mMiniLMv2-L12-H384-v1, chosen for interactive-latency scoring (Section 3.1). We additionally ran a targeted, smaller-scale check of sentence-selection agreement between this backend and the reference implementation's configurable BGE-reranker-v2-m3 alternative, on 20 of the 140 ARCD questions at $r = 0.5$: the two backends select substantially different sentence subsets (mean Jaccard overlap of the kept-sentence sets 0.449, median 0.429, with zero exact matches across the 20 questions), yet the gold-answer sentence survives pruning under both backends on 19 of 20 questions, and both backends agree on whether the answer survives on 19 of 20 questions. This check uses a substring match against the released gold answer as a proxy for answer preservation, not

the paper's own token-F1 scorer, and covers 20 of 140 questions at a single ratio rather than the full grid, so it establishes that selection-level backend disagreement exists and is not obviously fatal to answer preservation on this subset, without establishing that the two backends are equivalent for the paper's headline F1 result or its serving-cost picture (Section 4.6-4.7): a larger or more accurate reranker could plausibly still improve selection quality at the cost of higher per-sentence scoring latency, changing the throughput/latency trade-off, and that full comparison remains future work. We subsequently extended this check to the full 140-question ARCD evaluation set at all three ratios used throughout this paper, still using the same substring-match proxy for answer preservation rather than the token-F1 scorer: at $r = 0.3$, mean Jaccard overlap is 0.498 (median 0.500), mmarco-mMiniLMv2 retains the gold-answer substring on 136/140 questions and BGE-reranker-v2-m3 on 139/140, and the two backends agree on the gold-presence outcome on 137/140; at $r = 0.5$, mean Jaccard is 0.505 (median 0.500), gold retention is 138/140 and 139/140, and the backends agree on 139/140; at $r = 0.7$, mean Jaccard is 0.631 (median 0.622), gold retention is 138/140 and 139/140, and the backends agree on 139/140. This full-scale result is directionally consistent with the 20-question check above and substantially strengthens it in sample size, while carrying the same scope limit: it is still a selection-agreement and substring-based gold-retention check, not a re-run of the paper's actual token-F1 evaluation on the BGE-reranker-v2-m3 backend, which would additionally require regenerating all 420 answers with Llama-3.1-8B-Instruct against BGE-pruned contexts.

(5) Generalization. The principal fidelity and quality experiments use one instruction-tuned generator, Llama-3.1-8B-Instruct. A second generator, Qwen2.5-7B-Instruct [66], has now been tested under the identical ARCD protocol (Section 4.9): the $r = 0.3$ token-F1 advantage over naive truncation replicates in direction and significance, and at $r = 0.5$ it is significant under Qwen2.5 where it was not under Llama-3.1, so the central quality result is not an artifact of one generator family. This remains evidence from two related, similarly sized (7-8B parameter), instruction-tuned open-weight generators rather than a systematic study; whether the result extends to substantially larger or smaller models, closed models, or generators not instruction-tuned on comparable data is untested, and the end-to-end check still uses only one simple retriever (TF-IDF).

(5a) Arabic variety. Every experiment in this paper uses Modern Standard Arabic. The 30-pair tokenization corpus is MSA, ARCD's questions and Wikipedia source paragraphs are MSA, and the end-to-end retrieval corpus is built from the same ARCD paragraphs, so the title's and Abstract's references to Arabic should be read as MSA throughout. This matters more for Arabic than the generic caveat about language varieties would suggest: CAMeLBERT showed that matching the variant of Arabic seen in pretraining to the variant seen downstream affects performance more than raw pretraining corpus size does [41], and every component of LSPM that could be variety-sensitive is a component we did not vary. The sentence splitter keys on MSA and Latin punctuation and would segment dialectal text written with inconsistent or absent terminal punctuation poorly; the cross-encoder is a multilingual MS MARCO reranker whose Arabic training signal is overwhelmingly MSA; and the tokenization-disparity ratio itself would be expected to differ for dialectal orthography, which is less represented in tokenizer training corpora than MSA. Whether the $r = 0.3$ advantage survives on Egyptian, Levantine, Gulf, or Maghrebi Arabic, or on mixed-variety and code-switched text, is untested and should not be assumed.

(5b) Retrieval realism. The main ARCD re-test's candidate pool guarantees the gold paragraph's presence and draws its five distractors from unrelated articles (Section 3.4); this isolates post-retrieval sentence selection but is easier than a realistic setting where retrieval can fail and surviving distractors are often topically similar rather than unrelated. The results in Section 4.3 and Section 4.8 should accordingly be read as evidence about

selection quality under controlled candidate-pool construction, not as a demonstrated end-to-end RAG accuracy result under realistic full-corpus retrieval. The end-to-end check (Section 4.4) relaxes the retrieval-failure half of this concern with a real TF-IDF retriever, but its 234-paragraph corpus is small and a single lexical retriever does not supply the harder, topically similar negatives a larger corpus or a BM25, dense, or hybrid retriever would. Repeating the ARCD comparison over a realistic, full-corpus retriever with topically similar hard negatives is identified here as a specific, not-yet-run experiment rather than folded into the main result's claimed scope.

(6) Statistical scope. The significant result is concentrated at $r = 0.3$ and does not remain significant after correction at $r = 0.5$ or 0.7 . An independent replication of $r = 0.3$ on a fresh, disjoint sample has now been run (Section 4.10) and reproduces the finding with the same or greater statistical strength; testing across additional datasets and retrieval regimes beyond ARCD would still be needed before generalizing the result past the conditions actually tested here. Two reporting gaps belong here as well. No standardized effect size (for example a rank-biserial correlation accompanying each Wilcoxon test) was computed by the released analysis pipeline for the $n = 140$ contrasts, so the differences in Section 4.3 are reported as unstandardized paired differences with bootstrap intervals only, and cross-study comparison or meta-analysis of those contrasts is correspondingly harder; adding them is a mechanical improvement for a future version of the analysis code. And the TOST equivalence margin of ± 0.05 F1 is asserted rather than derived from a smallest-effect-size-of-interest argument, which the equivalence-testing literature we cite treats as the substantive step [69]; a reader who disagrees with that margin should discount the equivalence claims accordingly.

Taken together, these limitations mean that memory-efficient vLLM-based deployment remains this work's motivating design objective rather than a demonstrated result: it is now supported by a repeated benchmark showing a generation-only serving-cost result (Section 3.1) of zero measurable penalty relative to raw, rather than by a demonstrated memory-efficiency win at a realistic operating point; the context-selection result in Section 4.3 remains the part of this paper's contribution that is best supported. The GPU benchmark (Section 4.6) is now a real, repeated, randomized measurement that we do rely on, but for a deliberately narrow purpose: it establishes the absence of a throughput or TTFT penalty, and it does not establish a memory benefit, because the workload never approached cache saturation. A true end-to-end benchmark that includes the pruning step's own cost inside the live request path (Section 4.11) shows this generation-only result does not extend to concurrent-load online serving, where LSPM's throughput and completion rate degrade substantially as concurrency rises; and a matched controller-versus-fixed-ratio comparison under memory pressure (Section 4.12) finds a mixed serving-latency result with no answer-quality data, so neither open item is resolved in LSPM's favor. One qualification applies to the paper's strongest result and is stated here rather than buried in Methods. The $n = 140$ sample size was originally justified by a power calculation performed on the effect size observed in an earlier 40-question run, and those 40 questions are not a separate pilot but exactly the first 40 of the 140 (Section 3.4). That makes the justification circular, and observed power computed from one's own data is not a valid basis for asserting adequate power in the first place [70], so we no longer offer it: $n = 140$ is reported as a pragmatic sample size. The interpretive consequence is that the decision to extend the sample was taken after inspecting a subset of the same data, which is a data-dependent stopping rule and can inflate the Type I error rate of the resulting test to an extent this design does not let us quantify. We therefore do not present the original $n = 140$ result, taken alone, as an independently confirmed finding: it is a real, Holm-corrected, adequately intervalled result on 140 real questions with 980 real generations, and it is the best-supported claim in this paper, but on its own it remains a data-dependent

continuation of the pilot that set its sample size. What removes that qualification is a separate, independent replication on a fresh, disjoint sample of 140 ARCD questions sharing no question identifiers with the original set or its pilot (Section 4.10): the same $r = 0.3$ advantage over naive truncation, and the same equivalence of LSPM to the unpruned answer, both reproduce there with equal or greater statistical strength, on data that could not have influenced the original sample-size decision. Taken together, the original result and this replication constitute independent confirmation of the central $r = 0.3$ finding, even though the original $n = 140$ test alone does not.

6. Conclusions

Arabic RAG systems deployed on memory-efficient serving engines such as vLLM face a KV-cache pressure problem rooted in tokenization, confirmed with statistical significance across two independent tokenizers (aggregate ratio 1.69x-1.76x, $p < 0.0001$, $n = 30$ Modern Standard Arabic sentence pairs), in line with the cross-linguistic tokenizer-fertility literature. All results reported here are Modern Standard Arabic results; dialectal Arabic was not tested (Section 5). We introduced LSPM, a lightweight, sentence-level, cross-encoder-driven pruning middleware, and evaluated it with the same reporting standard applied regardless of outcome: a paired baseline comparison against naive sentence-count-matched truncation, re-tested on 140 real, ground-truth-scored ARCD questions, finds a statistically significant head-to-head advantage for LSPM at the most aggressive compression ratio tested ($r = 0.3$: mean token-F1 0.652 vs. 0.537, paired difference +0.114, bootstrap 95% CI +0.056 to +0.176, Holm-Bonferroni-adjusted $p = 0.0067$), where LSPM is additionally equivalent to the unpruned answer (F1 0.666; TOST, $n = 140$, $p = 0.037$) while naive truncation is significantly worse (adjusted $p = 0.0007$); at more moderate ratios the comparison remains open. A real TF-IDF end-to-end check ($n = 60$) and a blind two-rater human evaluation (90 answers, mean correctness 1.80 for LSPM vs. 1.72 for raw and 1.55 for truncation on a 0-2 scale) both directionally reinforce this result. Prompt-token counts recomputed offline for all 980 cells with the generator's own tokenizer show that LSPM achieves this while consuming roughly 10% fewer prompt tokens than the truncation baseline it beats, so the advantage is not a token-budget artifact; feeding those measured token reductions through Llama-3.1-8B-Instruct's published KV-cache geometry (32 layers $\times$ 8 key-value heads $\times$ head dimension 128 $\times$ 2 tensors $\times$ 2 bytes = 128 KiB per token) yields an analytical saving of 57.4 to 125.5 MiB of KV cache per request across the three ratios tested. A corrected GPU benchmark on a self-hosted vLLM server, designed so its output is the one to be trusted (35 method-by-concurrency cells $\times$ 3 repeats = 105 runs, 191,652 requests, zero failures, randomized cell order, pruning moved out of the load generator), finds a generation-only serving-cost result (Section 3.1) of zero measurable penalty relative to raw: LSPM's throughput is within 3% of unpruned serving at every concurrency level tested and slightly ahead at 100 concurrent clients (55.7 vs. 54.1 req/s, with lower median and p95 TTFT), which removes the apparent throughput deficit that the earlier confounded run had reported. We deliberately stop short of a memory-efficiency claim from that sweep alone: KV-cache occupancy never exceeded 8.8% of capacity in it, so the regime in which the analytical saving would matter was not entered there. A redesigned controller ablation, using real contexts and a deliberately constrained cache pool, subsequently did reach that regime, driving occupancy up to 99.2% and the controller's ratio across its full configured range (0.20-0.80); the adaptive mechanism is accordingly now shown to engage under genuine cache pressure, though the dynamic condition retained more context than a fixed $r = 0.5$ comparison throughout most of that run and was not faster on TTFT or latency, so whether adapting the ratio helps, rather than merely occurring, was left open at that point. A follow-up comparison against a fixed ratio matched to the controller's own realized mean, run under the same cache pressure (Section 4.12), has since closed that

question with a mixed result: the dynamic controller has lower mean TTFT and total latency, but a worse p95 TTFT and fewer completed requests than the matched fixed ratio, and no answer-quality data were collected for either arm, so adapting the ratio is not shown to be an improvement over a well-chosen fixed one. A separate true end-to-end online benchmark (Section 4.11), which scores and prunes context inside the live request path rather than reading it from a precomputed lookup, further finds that LSPM's throughput and completion rate degrade substantially relative to raw serving as concurrency rises, in contrast to the generation-only zero-measurable-penalty result reported above. We also closed the two most important remaining gaps: a token-budget-matched rerun, in which naive truncation receives the same per-question token budget LSPM actually spent, reproduces the $r = 0.3$ advantage (+0.140 F1, Holm-adjusted $p = 0.0003$), and a head-to-head comparison against LLMingua-2, a modern learned compressor, finds LSPM significantly ahead on token-F1 at every ratio tested, most convincingly at $r = 0.3$ and 0.5 (largest gap +0.211 at $r = 0.3$), with the $r = 0.7$ gap the weakest of the three and its bootstrap interval and rank-based test disagreeing at the margin (Section 4.8). The central $r = 0.3$ result also generalizes to a second generator, Qwen2.5-7B-Instruct, with the same or greater statistical strength than on the original generator. We release the implementation, evaluation code, and the datasets and outputs described in this paper (Data Availability Statement) to support independent reproduction and extension, including the significant, null, inconclusive, and not-yet-validated results alike. Finally, because the original $n = 140$ sample size was derived from an internal pilot that is a subset of the same 140 questions, we additionally ran an independent replication on 140 freshly sampled ARCD questions sharing no question identifiers with the original set (Section 4.10): the $r = 0.3$ advantage over naive truncation and the equivalence of LSPM to the unpruned answer both reproduce there with equal or greater statistical strength, so the central finding is now independently confirmed rather than resting on the original sample alone.

Supplementary Materials: The following supporting information can be downloaded at: <https://www.mdpi.com/article/10.3390/xxxxx/s1>, Table S1: full protocol and per-ratio ROUGE-L/BERTScore-F1 results for the $n = 8$ fidelity pilot (Section 4.2); Table S2: the full nine-test multiplicity-correction audit (raw and Holm-Bonferroni-adjusted p -values for every contrast in the nine-test family of Section 4.3); Table S3: full results of the original, superseded dynamic-controller ablation referenced in Section 4.7.

Author Contributions: Conceptualization, A.A.R.T.; methodology, A.A.R.T.; software, A.A.R.T.; validation, N.A.Z.; formal analysis, N.A.Z.; investigation, A.A.R.T.; resources, A.A.R.T. and N.A.Z.; data curation, N.A.Z.; writing - original draft preparation, A.A.R.T.; writing - review and editing, N.A.Z.; visualization, A.A.R.T. and N.A.Z.; supervision, N.A.Z.; project administration, N.A.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding. Inference costs for the hosted-API experiments were incurred against a free-tier allowance, and the GPU benchmark, controller ablation, true end-to-end online serving benchmark (Section 4.11), and controller-versus-matched-fixed-ratio comparison (Section 4.12) were run on rented single-GPU cloud instances at the authors' own expense.

Institutional Review Board Statement: Ethical review and approval by a formal committee were not required for this study. The only human involvement consisted of two adult volunteer bilingual annotators who rated already-generated machine text outputs for correctness and faithfulness; no clinical, physiological, or psychological intervention was performed on any participant, no personally identifiable, demographic, or otherwise sensitive data were collected, and no vulnerable population

was involved. This risk profile places the task outside the scope of formal ethics committee review under the authors' institutional policy, consistent with the treatment of minimal-risk, non-interventional annotation and evaluation tasks common in NLP research.

Informed Consent Statement: Informed consent was obtained verbally from both annotators involved in the study before the task began, covering the purpose of the task, its voluntary and unpaid nature, the right to withdraw at any time without consequence, and aggregate-only reporting of results. A separate written consent form was not administered because the task involved no more than minimal risk, collected no personally identifiable or sensitive information, and involved no deception; verbal consent, recorded by the researchers at the time, was judged sufficient documentation for a task of this risk profile.

Data Availability Statement: The datasets, evaluation outputs, GPU benchmark results, and implementation are publicly released at <https://github.com/Akramtaha98/vllm-arabic-rag>; the LSPM middleware, evaluation scripts, statistical analysis scripts, and benchmarking harness; the tokenization and fidelity/baseline pilot datasets; the ARCD ground-truth re-test dataset and code; and the end-to-end retrieval check dataset and code. For the corrected GPU benchmark of Section 4.6, the complete raw per-cell artifacts of all 105 runs are released under results/sweep_v2/rep1, rep2, and rep3 (one Locust statistics file, statistics history, failure and exception log, and per-second vLLM /metrics scrape per cell), together with the harness that produced them (precompute_contexts.py, run_full_sweep.py --repeats 3) and the analysis script (analyze_sweep_results.py) that aggregates them into the released summary (results/sweep_summary_v2.csv) and the comparison figure reproduced as Figure 2 (results/sweep_comparison_v2.png); every number reported in Section 4.6 and in Tables 3 and 4 is recomputable from the released raw artifacts by re-running that script. For the controller ablation of Section 4.7, the full per-request logs of both conditions are released under results/controller_ablation (fixed_ratio_log.jsonl, 2,294 requests; dynamic_ratio_log.jsonl, 2,302 requests), each row recording the ratio used, the KV-cache occupancy read at request time, TTFT, and total latency, alongside the harness (run_controller_ablation.py). For the redesigned controller ablation of Section 4.7, the full per-request logs of both conditions are released under results/controller_ablation_v2 (fixed_ratio_log.jsonl, 653 requests; dynamic_ratio_log.jsonl, 655 requests), alongside the harness that produced them (benchmark/run_controller_ablation_v2.py); the earlier, non-adaptive ablation's logs remain available unchanged under results/controller_ablation for comparison. For the true end-to-end online serving benchmark of Section 4.11, the per-cell summary statistics for all three ratios tested are released as results/online_e2e_r03/summary.csv, results/online_e2e_r05/summary.csv, and results/online_e2e_r07/summary.csv, alongside the harness that produced them (benchmark/run_online_e2e_benchmark.py). For the controller-versus-matched-fixed-ratio comparison of Section 4.12, the full per-request logs of both conditions are released as results/controller_vs_matched/dynamic_log.jsonl (5,637 requests) and results/controller_vs_matched/matched_fixed_log.jsonl (6,299 requests), each row recording the condition, ratio in effect, KV-cache occupancy at request time, TTFT, and total latency, alongside the harness that produced them (scripts/run_controller_vs_matched_fixed.py); this run used the harness's --skip-quality flag, so no answer-quality logs accompany it. The token-budget-matched and modern-compression-baseline comparisons of Section 4.8 are released as data/arcd_token_budget_results.jsonl and data/arcd_llmlingua2_results.jsonl (140 questions x 3 ratios each, 420 rows), alongside the harnesses that produced them (scripts/run_arcd_token_budget.py, scripts/run_arcd_llmlingua2_baseline.py), and the Qwen2.5-7B-Instruct generalization check of Section 4.9 is released as data/arcd_qwen25_results.jsonl (140 questions x 7 conditions, 980 rows, scripts/run_arcd_qwen25_baseline.py). The fresh-seed disjoint replication of Section 4.10 is released as data/arcd_eval_set_replication.json (the 140 freshly sampled

1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714

questions, verified to share no question identifiers with the original set) and data/arcd_replication_results.jsonl (140 questions x 3 conditions, 420 rows), together with the scripts that produced them (scripts/build_arcd_eval_set_replication.py, scripts/run_arcd_replication.py, scripts/score_arcd_replication.py). Every Exact Match and token-F1 figure in this paper, including Table 2, is computed with one scorer, released as scripts/score_arcd.py; a differently named scorer, eval/arabic_em_f1.py, is referenced by some internal analysis scripts but is not present in this released repository. The prompt-token counts of Section 4.3 are released as results/arcd_prompt_tokens.jsonl (one row per each of the 980 cells, recording context characters, retained sentences, context tokens, and full prompt tokens) together with the offline replay script that produced them (compute_prompt_tokens.py, which also re-verifies every reconstructed context against the character and sentence counts recorded by the live run) and the summary script that derives the reported means, paired differences, bootstrap intervals, and KV-cache byte projection (summarize_prompt_tokens.py). The repository also still contains the aggregated summary of the earlier, superseded single-run sweep (results/sweep_summary.csv); its per-request logs were generated on a temporary rented cloud instance and were not preserved, which is why this manuscript no longer draws on it for any claim. This repository was tagged and archived on Zenodo (version DOI: 10.5281/zenodo.22846964; concept DOI, always resolving to the latest version: 10.5281/zenodo.21936820).

1715
1716
1717
1718
1719
1720

Acknowledgments: During the preparation of this manuscript, the authors used Anthropic Claude for the purposes of software implementation assistance (the LSPM middleware, evaluation scripts, statistical analysis scripts), literature-search assistance in identifying candidate related work subsequently verified against primary sources, and drafting assistance on this manuscript. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

1721

Conflicts of Interest: The authors declare no conflicts of interest.

1722

Abbreviations

1723
1724
1725
1726
1727
1728

The following abbreviations are used in this manuscript:
RAG: Retrieval-Augmented Generation; LLM: Large Language Model; KV: Key-Value; LSPM: Lightweight Semantic Pruning Middleware; ARCD: Arabic Reading Comprehension Dataset; EM: Exact Match; F1: F1 score (harmonic mean of precision and recall); TOST: Two One-Sided Tests (equivalence test); TTFT: Time to First Token; TF-IDF: Term Frequency-Inverse Document Frequency.

1729

References

1730 1. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, L.; Polosukhin, I. Attention Is All You Need.
1731 In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2017.
1732 <https://doi.org/10.48550/arXiv.1706.03762>.
1733 2. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Kuttler, H.; Lewis, M.; Yih, W.; Rocktaschel, T.; et al.
1734 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Proceedings of the Advances in Neural Information
1735 Processing Systems (NeurIPS), 2020. <https://doi.org/10.48550/arXiv.2005.11401>.
1736 3. Brown, T.B.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et
1737 al. Language Models are Few-Shot Learners. In Proceedings of the Advances in Neural Information Processing Systems
1738 (NeurIPS), 2020. <https://doi.org/10.48550/arXiv.2005.14165>.
1739 4. Kwon, W.; Li, Z.; Zhuang, S.; Sheng, Y.; Zheng, L.; Yu, C.H.; Gonzalez, J.; Zhang, H.; Stoica, I. Efficient Memory Management
1740 for Large Language Model Serving with PagedAttention. In Proceedings of the 29th ACM Symposium on Operating Systems
1741 Principles (SOSP), 2023. <https://doi.org/10.1145/3600006.3613165>.
1742 5. Xiao, G.; Tian, Y.; Chen, B.; Han, S.; Lewis, M. Efficient Streaming Language Models with Attention Sinks. In Proceedings of the
1743 International Conference on Learning Representations (ICLR), 2024. <https://doi.org/10.48550/arXiv.2309.17453>.

6. Zhang, Z.; Sheng, Y.; Zhou, T.; Chen, T.; Zheng, L.; Cai, R.; Song, Z.; Tian, Y.; Re, C.; Barrett, C.; et al. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2023. <https://doi.org/10.52202/075280-1506>.
7. Sennrich, R.; Haddow, B.; Birch, A. Neural Machine Translation of Rare Words with Subword Units. In Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL), 2016, pp. 1715-1725. <https://doi.org/10.18653/v1/P16-1162>.
8. Jiang, H.; Wu, Q.; Lin, C.-Y.; Yang, Y.; Qiu, L. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. <https://doi.org/10.18653/v1/2023.emnlp-main.825>.
9. Jiang, H.; Wu, Q.; Luo, X.; Li, D.; Lin, C.-Y.; Yang, Y.; Qiu, L. LongLLMingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression. arXiv 2023, arXiv:2310.06839. <https://doi.org/10.48550/arXiv.2310.06839>.
10. Li, Y.; Dong, B.; Lin, C.; Guerin, F. Compressing Context to Enhance Inference Efficiency of Large Language Models. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. <https://doi.org/10.18653/v1/2023.emnlp-main.391>.
11. Xu, F.; Shi, W.; Choi, E. RECOMP: Improving Retrieval-Augmented LMs with Compression and Selective Augmentation. In Proceedings of the International Conference on Learning Representations (ICLR), 2024. <https://doi.org/10.48550/arXiv.2310.04408>.
12. Chevalier, A.; Wettig, A.; Ajith, A.; Chen, D. Adapting Language Models to Compress Contexts. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. <https://doi.org/10.48550/arXiv.2305.14788>.
13. Mu, J.; Li, X.L.; Goodman, N. Learning to Compress Prompts with Gist Tokens. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2023. <https://doi.org/10.48550/arXiv.2304.08467>.
14. Liu, N.F.; Lin, K.; Hewitt, J.; Paranjape, A.; Bevilacqua, M.; Petroni, F.; Liang, P. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguist.* 2024, 12, 157-173. https://doi.org/10.1162/tacl_a_00638.
15. Ahia, O.; Kumar, S.; Gonen, H.; Kasai, J.; Mortensen, D.; Smith, N.A.; Tsvetkov, Y. Do All Languages Cost the Same? Tokenization in the Era of Commercial Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023; pp. 9904-9923. <https://doi.org/10.18653/v1/2023.emnlp-main.614>.
16. Al-Khalifa, S.; Durrani, N.; Al-Khalifa, H.; Alam, F. The Landscape of Arabic Large Language Models (ALLMs): A New Era for Arabic Language Technology. *Commun. ACM* 2025. <https://doi.org/10.1145/3737453>.
17. Alkaoud, M.; Alsaqoub, M.; Aljodhi, I.; Alqadibi, A.; Altammami, O. ACLM: Developing a Compact Arabic Language Model. *Int. Arab J. Inf. Technol.* 2025, 22, 535-546. <https://doi.org/10.34028/iajit/22/3/9>.
18. Nogueira, R.; Cho, K. Passage Re-ranking with BERT. arXiv 2019, arXiv:1901.04085. <https://doi.org/10.48550/arXiv.1901.04085>.
19. Rust, P.; Pfeiffer, J.; Vulic, I.; Ruder, S.; Gurevych, I. How Good is Your Tokenizer? On the Monolingual Performance of Multilingual Language Models. In Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP), 2021; pp. 3118-3135. <https://doi.org/10.18653/v1/2021.acl-long.243>.
20. Petrov, A.; La Malfa, E.; Torr, P.H.S.; Bibi, A. Language Model Tokenizers Introduce Unfairness Between Languages. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2023. <https://doi.org/10.48550/arXiv.2305.15425>.
21. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv 2023, arXiv:2312.10997. <https://doi.org/10.48550/arXiv.2312.10997>.
22. Guu, K.; Lee, K.; Tung, Z.; Pasupat, P.; Chang, M.-W. REALM: Retrieval-Augmented Language Model Pre-Training. In Proceedings of the 37th International Conference on Machine Learning (ICML), 2020. <https://doi.org/10.48550/arXiv.2002.08909>.
23. Khandelwal, U.; Levy, O.; Jurafsky, D.; Zettlemoyer, L.; Lewis, M. Generalization through Memorization: Nearest Neighbor Language Models. In Proceedings of the International Conference on Learning Representations (ICLR), 2020. <https://doi.org/10.48550/arXiv.1911.00172>.
24. Izacard, G.; Grave, E. Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering. In Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2021, pp. 874-880. <https://doi.org/10.48550/arXiv.2007.01282>.
25. Ram, O.; Levine, Y.; Dalmedigos, I.; Muhlgay, D.; Shashua, A.; Leyton-Brown, K.; Shoham, Y. In-Context Retrieval-Augmented Language Models. *Trans. Assoc. Comput. Linguist.* 2023, 11, 1316-1331. https://doi.org/10.1162/tacl_a_00605.

-
26. Asai, A.; Wu, Z.; Wang, Y.; Sil, A.; Hajishirzi, H. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In Proceedings of the International Conference on Learning Representations (ICLR), 2024. <https://doi.org/10.48550/arXiv.2310.11511>.
27. Jeong, S.; Baek, J.; Cho, S.; Hwang, S.J.; Park, J.C. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2024. <https://doi.org/10.48550/arXiv.2403.14403>.
28. Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N.A.; Lewis, M. Measuring and Narrowing the Compositionality Gap in Language Models. In Findings of the Association for Computational Linguistics: EMNLP 2023. <https://doi.org/10.48550/arXiv.2210.03350>.
29. Robertson, S.; Zaragoza, H. The Probabilistic Relevance Framework: BM25 and Beyond. *Found. Trends Inf. Retr.* 2009, 3, 333-389. <https://doi.org/10.1561/1500000019>.
30. Karpukhin, V.; Oguz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W. Dense Passage Retrieval for Open-Domain Question Answering. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020. <https://doi.org/10.18653/v1/2020.emnlp-main.550>.
31. Khattab, O.; Zaharia, M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, 2020. <https://doi.org/10.1145/3397271.3401075>.
32. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019. <https://doi.org/10.18653/v1/D19-1410>.
33. Chen, J.; Xiao, S.; Zhang, P.; Luo, K.; Lian, D.; Liu, Z. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv* 2024, [arXiv:2402.03216](https://doi.org/10.48550/arXiv.2402.03216).
34. Wang, L.; Yang, N.; Huang, X.; Yang, L.; Majumder, R.; Wei, F. Multilingual E5 Text Embeddings: A Technical Report. *arXiv* 2024, [arXiv:2402.05672](https://doi.org/10.48550/arXiv.2402.05672).
35. Zhang, X.; Thakur, N.; Ogundepo, O.; Kamalloo, E.; Alfonso-Hermelo, D.; Li, X.; Liu, Q.; Rezagholizadeh, M.; Lin, J. MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages. *Trans. Assoc. Comput. Linguist.* 2023, 11, 1114-1131. https://doi.org/10.1162/tacl_a_00595.
36. Muennighoff, N.; Tazi, N.; Magne, L.; Reimers, N. MTEB: Massive Text Embedding Benchmark. In Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL), 2023. <https://doi.org/10.18653/v1/2023.eacl-main.148>.
37. Johnson, J.; Douze, M.; Jegou, H. Billion-Scale Similarity Search with GPUs. *IEEE Trans. Big Data* 2021, 7, 535-547. <https://doi.org/10.1109/TBDATA.2019.2921572>.
38. Antoun, W.; Baly, F.; Hajj, H. AraBERT: Transformer-based Model for Arabic Language Understanding. In Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools (OSACT), 2020. <https://doi.org/10.48550/arXiv.2003.00104>.
39. Antoun, W.; Baly, F.; Hajj, H. AraGPT2: Pre-Trained Transformer for Arabic Language Generation. In Proceedings of the 6th Arabic Natural Language Processing Workshop (WANLP), 2021. <https://doi.org/10.48550/arXiv.2012.15520>.
40. Nagoudi, E.M.B.; Elmadany, A.; Abdul-Mageed, M. AraT5: Text-to-Text Transformers for Arabic Language Generation. In Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2022, pp. 628-647. <https://doi.org/10.48550/arXiv.2109.12068>.
41. Inoue, G.; Alhafni, B.; Baimukan, N.; Bouamor, H.; Habash, N. The Interplay of Variant, Size, and Task Type in Arabic Pre-trained Language Models. In Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP), 2021. <https://doi.org/10.48550/arXiv.2103.06678>.
42. Obeid, O.; Zalmout, N.; Khalifa, S.; Taji, D.; Oudah, M.; Alhafni, B.; Inoue, G.; Eryani, F.; Erdmann, A.; Habash, N. CAMEL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing. In Proceedings of the 12th Language Resources and Evaluation Conference (LREC), 2020, pp. 7022-7032. <https://doi.org/10.63317/295vncriwcn2>.
43. Safaya, A.; Abdullatif, M.; Yuret, D. KUISAIL at SemEval-2020 Task 12: BERT-CNN for Offensive Speech Identification in Social Media. In Proceedings of the Fourteenth Workshop on Semantic Evaluation (SemEval-2020), 2020. <https://doi.org/10.48550/arXiv.2007.13184>.
44. Elmadany, A.; Nagoudi, E.M.B.; Abdul-Mageed, M. ORCA: A Challenging Benchmark for Arabic Language Understanding. In Findings of the Association for Computational Linguistics: ACL 2023. <https://doi.org/10.48550/arXiv.2212.10758>.

-
45. Seelawi, H.; Tuffaha, I.; Gzawi, M.; Farhan, W.; Talafha, B.; Badawi, R.; Sober, Z.; Al-Dweik, O.; Freihat, A.A.; Al-Natsheh, H. ALUE: Arabic Language Understanding Evaluation. In Proceedings of the Sixth Arabic Natural Language Processing Workshop (WANLP), 2021, pp. 173-184. Available online: <https://aclanthology.org/2021.wanlp-1.18/> (accessed on 26 August 2026).
46. Koto, F.; Li, H.; Shatnawi, S.; Doughman, J.; Sadallah, A.B.; Alraeesi, A.; Almubarak, K.; Alyafeai, Z.; Sengupta, N.; Shehata, S.; et al. ArabicMMLU: Assessing Massive Multitask Language Understanding in Arabic. In Findings of the Association for Computational Linguistics: ACL 2024. <https://doi.org/10.18653/v1/2024.findings-acl.334>.
47. Huang, H.; Yu, F.; Zhu, J.; Sun, X.; Cheng, H.; Dingjie, S.; Chen, Z.; Alharthi, M.; An, B.; He, J.; et al. AceGPT, Localizing Large Language Models in Arabic. In Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2024, pp. 8139-8163. <https://doi.org/10.18653/v1/2024.naacl-long.450>.
48. Bandarkar, L.; Liang, D.; Muller, B.; Artetxe, M.; Shukla, S.N.; Husa, D.; Goyal, N.; Krishnan, A.; Zettlemoyer, L.; Khabsa, M. The Belebele Benchmark: A Parallel Reading Comprehension Dataset in 122 Language Variants. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024. <https://doi.org/10.48550/arXiv.2308.16884>.
49. Conneau, A.; Lample, G.; Rinott, R.; Williams, A.; Bowman, S.R.; Schwenk, H.; Stoyanov, V. XNLI: Evaluating Cross-Lingual Sentence Representations. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2018. <https://doi.org/10.48550/arXiv.1809.05053>.
50. Mozannar, H.; El Hajal, K.; Maamary, E.; Hajj, H. Neural Arabic Question Answering. In Proceedings of the Fourth Arabic Natural Language Processing Workshop (WANLP), 2019. <https://doi.org/10.18653/v1/W19-4612>.
51. Rajpurkar, P.; Zhang, J.; Lopyrev, K.; Liang, P. SQuAD: 100,000+ Questions for Machine Comprehension of Text. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2016. <https://doi.org/10.48550/arXiv.1606.05250>.
52. Yu, G.-I.; Jeong, J.S.; Kim, G.-W.; Kim, S.; Chun, B.-G. Orca: A Distributed Serving System for Transformer-Based Generative Models. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2022, pp. 521-538. Available online: <https://www.usenix.org/conference/osdi22/presentation/yu> (accessed on 26 August 2026).
53. Agrawal, A.; Kedia, N.; Panwar, A.; Mohan, J.; Kwatra, N.; Gulavani, B.S.; Tumanov, A.; Ramjee, R. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2024. <https://doi.org/10.48550/arXiv.2403.02310>.
54. Dao, T.; Fu, D.Y.; Ermon, S.; Rudra, A.; Re, C. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), 2022. <https://doi.org/10.52202/068431-1189>.
55. Shazeer, N. Fast Transformer Decoding: One Write-Head is All You Need. arXiv 2019, arXiv:1911.02150. <https://doi.org/10.48550/arXiv.1911.02150>.
56. Ainslie, J.; Lee-Thorp, J.; de Jong, M.; Zemlyanskiy, Y.; Lebron, F.; Sanghai, S. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. In Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023. <https://doi.org/10.48550/arXiv.2305.13245>.
57. Papineni, K.; Roukos, S.; Ward, T.; Zhu, W.-J. BLEU: A Method for Automatic Evaluation of Machine Translation. In Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311-318. <https://doi.org/10.3115/1073083.1073135>.
58. Lin, C.-Y. ROUGE: A Package for Automatic Evaluation of Summaries. In Text Summarization Branches Out: Proceedings of the ACL Workshop, 2004, pp. 74-81. Available online: <https://aclanthology.org/W04-1013/> (accessed on 26 August 2026).
59. Zhang, T.; Kishore, V.; Wu, F.; Weinberger, K.Q.; Artzi, Y. BERTScore: Evaluating Text Generation with BERT. In Proceedings of the International Conference on Learning Representations (ICLR), 2020. <https://doi.org/10.48550/arXiv.1904.09675>.
60. Es, S.; James, J.; Espinosa Anke, L.; Schockaert, S. RAGAS: Automated Evaluation of Retrieval Augmented Generation. In Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL), 2024. <https://doi.org/10.18653/v1/2024.eacl-demo.16>.
61. Efron, B.; Tibshirani, R.J. An Introduction to the Bootstrap; Chapman & Hall/CRC: New York, NY, USA, 1993. <https://doi.org/10.1201/9780429246593>.
62. Wang, Z.; Araki, J.; Jiang, Z.; Parvez, M.R.; Neubig, G. Learning to Filter Context for Retrieval-Augmented Generation. arXiv 2023, arXiv:2311.08377. <https://doi.org/10.48550/arXiv.2311.08377>.

63. Hwang, T.; Jeong, S.; Cho, S.; Han, S.; Park, J. DSLR: Document Refinement with Sentence-Level Re-ranking and Reconstruction to Enhance Retrieval-Augmented Generation. In Proceedings of the 3rd Workshop on Knowledge Augmented Methods for NLP (KnowledgeNLP), 2024, pp. 73-92. <https://doi.org/10.18653/v1/2024.knowledgenlp-1.6>.
64. Mohamed, Y.; Elhoseiny, M.; Formal, T.; Chirkova, N. XProvenance: Zero-Cost Multilingual Context Pruning for Retrieval-Augmented Generation. In Proceedings of the European Conference on Information Retrieval (ECIR), 2026. <https://doi.org/10.48550/arXiv.2601.18886>.
65. Cheng, X.; Wang, X.; Zhang, X.; Ge, T.; Chen, S.-Q.; Wei, F.; Zhang, H.; Zhao, D. xRAG: Extreme Context Compression for Retrieval-Augmented Generation with One Token. arXiv 2024, arXiv:2405.13792. <https://doi.org/10.48550/arXiv.2405.13792>.
66. Qwen Team. Qwen2.5 Technical Report. arXiv 2024, arXiv:2412.15115. <https://doi.org/10.48550/arXiv.2412.15115>.
67. AI@Meta. The Llama 3 Herd of Models. arXiv 2024, arXiv:2407.21783. <https://doi.org/10.48550/arXiv.2407.21783>.
68. Schuirmann, D.J. A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability. *J. Pharmacokinet. Biopharm.* 1987, 15, 657-680. <https://doi.org/10.1007/BF01068419>.
69. Lakens, D. Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses. *Soc. Psychol. Personal. Sci.* 2017, 8, 355-362. <https://doi.org/10.1177/1948550617697177>.
70. Hoenig, J.M.; Heisey, D.M. The Abuse of Power: The Pervasive Fallacy of Power Calculations for Data Analysis. *Am. Stat.* 2001, 55, 19-24. <https://doi.org/10.1198/000313001300339897>.
71. Pan, Z.; Wu, Q.; Jiang, H.; Xia, M.; Luo, X.; Zhang, J.; Lin, Q.; Ruhle, V.; Yang, Y.; Lin, C.-Y.; et al. LLMingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression. In Findings of the Association for Computational Linguistics: ACL 2024, pp. 963-981. <https://doi.org/10.18653/v1/2024.findings-acl.57>.
72. Zhao, Y.; Li, Z.; Zhao, H.; Qi, B.; Guoming, L. DAC: A Dynamic Attention-aware Approach for Task-Agnostic Prompt Compression. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 19395-19407. <https://doi.org/10.18653/v1/2025.acl-long.952>.
73. Guo, S.; Zhang, S.; Ren, Z. Enhancing RAG Efficiency with Adaptive Context Compression. In Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 24061-24076. <https://doi.org/10.18653/v1/2025.findings-emnlp.1307>.
74. Frantar, E.; Ashkboos, S.; Hoefler, T.; Alistarh, D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. In Proceedings of the International Conference on Learning Representations (ICLR), 2023. <https://doi.org/10.48550/arXiv.2210.17323>.
75. Lin, J.; Tang, J.; Tang, H.; Yang, S.; Chen, W.-M.; Wang, W.-C.; Xiao, G.; Dang, X.; Gan, C.; Han, S. AWQ: Activation-Aware Weight Quantization for LLM Compression and Acceleration. In Proceedings of Machine Learning and Systems (MLSys), 2024. <https://doi.org/10.48550/arXiv.2306.00978>.
76. Hooper, C.; Kim, S.; Mohammadzadeh, H.; Mahoney, M.W.; Shao, Y.S.; Keutzer, K.; Gholami, A. KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization. arXiv 2024, arXiv:2401.18079. <https://doi.org/10.48550/arXiv.2401.18079>.
77. Leviathan, Y.; Kalman, M.; Matias, Y. Fast Inference from Transformers via Speculative Decoding. In Proceedings of the 40th International Conference on Machine Learning (ICML), 2023. <https://doi.org/10.48550/arXiv.2211.17192>.
78. Cai, T.; Li, Y.; Geng, Z.; Peng, H.; Lee, J.D.; Chen, D.; Dao, T. Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads. arXiv 2024, arXiv:2401.10774. <https://doi.org/10.48550/arXiv.2401.10774>.
79. Li, Y.; Wei, F.; Zhang, C.; Zhang, H. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. arXiv 2024, arXiv:2401.15077. <https://doi.org/10.48550/arXiv.2401.15077>.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.